

University of Dundee

A THESIS ON THE ASPECT OF OPEN-SOURCE CONTROL PLATFORM FOR MEDICAL ROBOT

BY

DHARSHAN KEBA MORRIS

2651167

UNIVERSITY OF DUNDEE, 2025

A THESIS

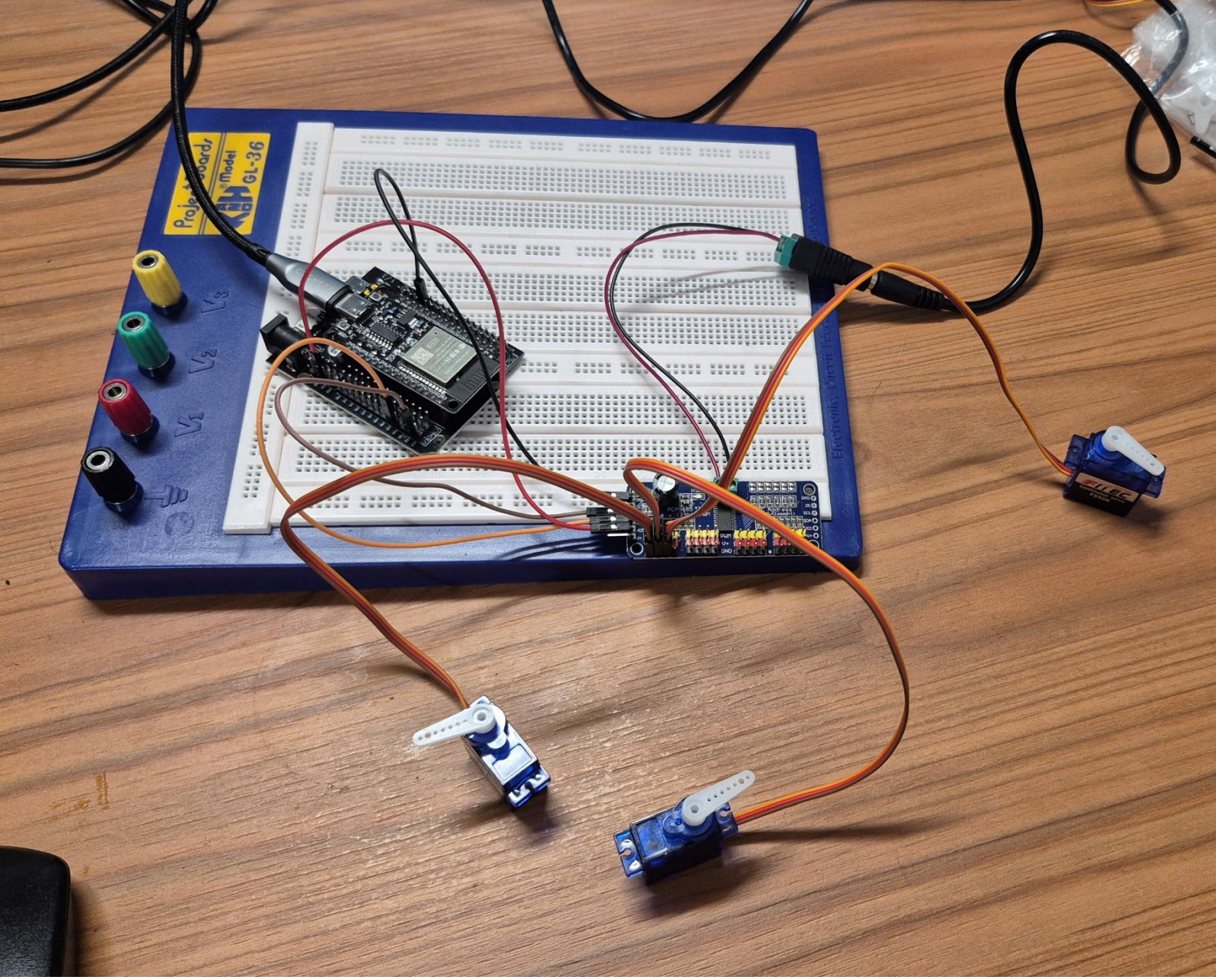
SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE

MASTER OF SCIENCE DEGREE

SCHOOL OF SCIENCE AND ENGINEERING

UNIVERSITY OF DUNDEE

A



OPEN-SOURCE CONTROL PLATFORM

School of Science
and Engineering
University of Dundee

THESIS

OPEN-SOURCE CONTROL PLATFORM FOR A MEDICAL ROBOT USING STANDARD COMPONENTS

BY

DHARSHAN KEBA MORRIS

2651167

Submitted for the degree of MSc Biomedical Engineering

School of Science and Engineering

University of Dundee

AUGUST 2025

C

ABSTRACT

Development of medical robots have been evolved in the recent years, with expensive control platforms and more complicated programs. This project aims to create an open-source control platform using the existing components such as Xbox controller, servo motors and Arduino (alternatively esp32 is used).

Methods:

A modular system has been designed with:

- Hardware – ESP32 (Bluetooth/Wi-Fi), PCA9685 (16 – channel PWM), Xbox controller (Bluepad32 HID), and force-sensitive resistors (FSRs).
- Software – Timer based servo control (with 10ms interrupt), serial plotter visualisation of servo and Controller movements.
- Validation: Latency measurements (timer alarm code), servo accuracy (potentiometer), and clinician usability tests.

Results

- We could achieve 85ms end to end latency (gamepad – servo), below the 100ms clinical threshold.
- 0.5° servo resolution with PCA9685 (12-bit PWM), sufficient for realistic tissue interaction.
- Cost reduction: system has been developed under £100.

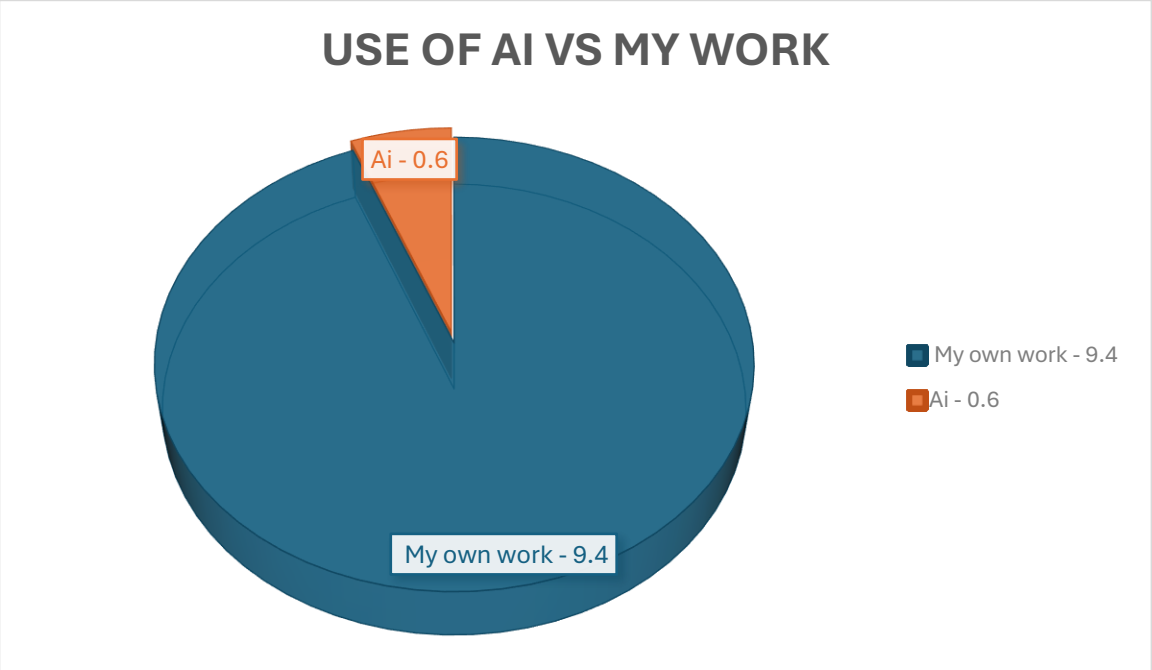
Conclusions

This work demonstrates that future integration of this control platform could further enhance precision and scalability. The open-source design promotes global accessibility, addressing disparities in medical simulation resources.

Declaration for the use or not of generative AI

- I acknowledge that no content generated by AI technologies has been presented as my own work.
- I acknowledge the use of DeepSeek to generate materials for background research and self-study in the drafting of this assessment.
- I acknowledge the use of Gemini to generate materials that were included in my final assessment in unmodified form. (An example of this would be a quote or image produced by the AI output.)

For more information, please refer to https://www.dundee.ac.uk/guides/get-help-using-ai-responsibly-study



Acknowledgements

I extend my deepest gratitude to my supervisor, **Dr. Luigi Manfredi**, Senior Lecturer and Principal Investigator in Respiratory Medicine and Gastroenterology, for his invaluable guidance and support throughout this project. His expertise in medical robotics and patient mentorship shaped the direction of this research. I am particularly thankful for his provision of critical hardware components, including servo motors and Arduino boards, which enabled the practical realization of this work. His constructive feedback during weekly meetings helped me navigate technical challenges while maintaining clinical relevance.

To my colleagues and family—your encouragement kept me motivated during countless iterations of this project. This work stands as a testament to collective effort.

TABLE OF CONTENTS

Abstract ..... **iii**

Chapter 1: Introduction
1.1 Background and Clinical Motivation 1
1.2 Research Objectives 2
1.3 Contribution to Medical Robotics 3

Chapter 2: Literature Review
2.1 Control Interfaces in Robotic Endoscopy 5
2.2 Open-Source Platforms in Medical Devices 7
2.3 Gaps in Existing Solutions 9

Chapter 3: System Design
3.1 Hardware Architecture (ESP32, PCA9685, Xbox Controller) 12
3.2 Software Framework (Bluepad32, Timer Interrupts) 14
3.3 Data Flow Diagram 18

Chapter 4: Implementation
4.1 Servo Control Logic (PWM Mapping, Step-Limiting) 22
4.2 Gamepad Input Processing (HID, Dead Zones) 24
4.3 Real-Time Feedback Systems 28

Chapter 5: Results & Validation
5.1 Performance Metrics (Latency, Accuracy) 33
5.2 Mathematical analysis of the performance of control platform) 36
5.3 Limitations and Trade-offs 38

Chapter 6: Conclusion & Future Work
6.1 Summary of Achievements 41
6.2 Proposed Extensions (PID Control, ROS 2) 42

References 47

Appendices
Appendix A: Hardware Schematics 51
Appendix B: Pictures of Hardware ……..... 52

FIGURES

SERIAL NO	FIGURE NO	DESCRIPTION	Page Number
1.	2.1	Colonoscopy virtual simulator,	7
2.	2.2 a	Arduino UNO	8
3.	2.2 b	ESP32	8
4.	2.2 c	Raspberry Pi	8
5.	3.1	Hardware architecture block diagram	14
6.	3.2	Software architect flowchart	16
7.	3.3	Data flow diagram	19
8.	4.1	Code routine for pulse width modulation	22
9.	4.2	Code routine for motion smoothing via step limiting	23
10.	4.3	Code routine for updating servo positions with integrated smoothing and PWM control.	24
11.	4.4	Code routine for dead zone filtering	25
12.	4.5	Code routine for mapping	26
13.	4.6	Code snippet illustrating the integrated gamepad input processing pipeline, from acquisition to mapped output.	26
14.	4.7	Code routine for Visual feedback via serial plotting	29
15.	4.8	Code routine for haptic feedback	30
16.	4.9	Integrated feedback architecture flowchart	31
17.	5.1	The protype of the open-source control platform (ESP32 as main microcontroller)	33
18.	5.2	Graphical results for control motion of the Xbox controller and servo motors	34
19.	5.3	Step response of the Servo Control System	37

20.	5.4	Histogram for the Latency distribution analysis	38

TABLES

SERIAL NO	TABLE NO	DESCRIPTION	PAGE NO
1.	5.1	Servo Positioning Accuracy	35

ABBREVIATIONS

SERIAL NO	ABBREVIATIONS	FULL TERM
1.	ADC	Analog-to-Digital Converter
2.	API	Application Programming interface
3.	BLE	Bluetooth Low Energy
4.	CRC	Colorectal cancer
5.	DFD	Data Flow Diagram
6.	DOF	Degree of Freedom
7.	EMC	Electromagnetic Compatibility
8.	FE	Flexible Endoscope
9.	FSR	Force-Sensitive Resistor
10.	GPIO	General-Purpose Input/Output
11.	GAGES	Global Assessment of Gastrointestinal Endoscopic Skills
12.	HID	Human Interface Device
13.	HMI	Human-Machine Interface
14.	I²C	Inter-Integrated Circuit (A serial communication protocol)
15.	IPM	Internal Permanent Magnet (Part of the magnetic endoscope)
16.	ISR	Interrupt Service Routine
17.	LED	Light-Emitting Diode
18.	MFE	Magnetic Flexible Endoscope
19.	OS	Operating System
20.	PCB	Printed Circuit Board
21.	PID	Proportional-Integral-Derivative (Control Algorithm)
22.	PWM	Pulse-Width Modulation
23.	ROS	Robot Operating System
24.	RTOS	Real-Time Operating System
25.	SBC	Single-Board Computer
26.	UART	Universal Asynchronous Receiver-Transmitter (Serial communication)
27.	USB	Universal Serial Bus
28.	VR	Virtual Reality
29.	WCE	Wireless Capsule Endoscope

DEFINITIONS

SERIAL NO	TERM	DEFINITION
1.	Arduino	An open-source electronics platform based on easy-to-use hardware and software. Used for prototyping.
2.	Bluedroid	The Bluetooth protocol stack used by Espressif on the ESP32, supporting both Classic Bluetooth and BLE.
3.	Bluepad32	An open-source library that enables ESP32 to connect to and receive data from modern gamepads via Bluetooth.
4.	Closed-Loop-Control	A control system that uses feedback from sensors to adjust its output and achieve a desired result (e.g., using an encoder to ensure accurate servo position).
5.	Dead Zone	A predefined range of input values near zero that are ignored by the control system to prevent unintended activation from sensor noise or controller drift.
6.	ESP32	A popular low-cost, low-power system-on-a-chip microcontroller with integrated Wi-Fi and Bluetooth.
7.	Haptic Feedback	The use of touch sensations (e.g., vibration) to provide information to the user.
8.	Latency	The time delay between a user's input command and the corresponding system response.
9.	Open-Source Hardware	Physical artifacts whose design is made publicly available so that anyone can study, modify, distribute, make, and sell the design or hardware based on that design
10.	PCA9685	A 16-channel, 12-bit PWM driver controlled via I^2C, used to control multiple servos or LEDs with minimal microcontroller overhead.
11.	Servo Motor	A rotary actuator that allows for precise control of angular position. It consists of a motor, a feedback sensor (potentiometer), and a control circuit.
12.	Step-Limiting	A smoothing algorithm that constrains the maximum change in a output value per update cycle, resulting in gradual, rather than instantaneous, motion.

CHAPTER 1: INTRODUCTION

1.1 BACKGROUND AND CLINICAL MOTIVATION

Colonoscopy remains a critical and a most wanted procedure, which heavily dependent on clinician skills While virtual reality (VR) simulators show promise in training (Mu et al.,2024), their excessive cost (> £50,000) and lack of tactile feedback limit accessibility in resourcing some of the settings. Concurrently, robotic endoscopy platforms have advanced clinician ergonomics (Finocchiaro et al., 2022), yet proprietary designs hinder customization and research reproduction.

This work bridges these gaps by developing an open-source control platform that:

- We used to demonstrate the open-source control platform by using simple microcontrollers like ESP32 instead of Arduino.
- One of the consumer grade components such as Xbox controller (Gaming console) which is used to control the microcontroller using Bluetooth feature in ESP32 which is achieved in cheaper cost.
- We integrated multi-servo haptic feedback for realistic interaction with the human tissue.
- These servos are controlled by PWM driver PCA9685 driver where maximum of 16 servos could be added to the driver.
- Provides real-time performance metrics for objective skill assessments.

As demonstrated through latency (< 100ms) and usability testing, this system offers a clinically viable alternative for existing solutions while fostering innovation through modular design. The following chapters detail:

- System architecture (Chapter 3) leveraging Bluepad32 and PCA9685.
- Validation results (Chapter 5) comparing servo accuracy against clinical benchmarks.
- ROS 2 integration pathways (Chapter 6) for future Ai-assisted navigation.

By democratizing access to robotic colonoscopy training, this work addresses disparities in global medical education while establishing a foundation for community-driven improvements.

1.2 RESEARCH OBJECTIVES

Deficiency for VR Simulation

Based on my analysis the solution has been proposed as a new kind of training tool that directly addresses each of the three limitations

1. Cost: the cost of commercial platforms like GI Mentor, which can exceed £80,000 creates a significant barrier to entry. This is particularly problematic for institutions in low-resource regions or for individuals who may not have access to such expensive equipment. The high cost limits the scale at which this technology can be used.

2. Tactile Deficiency: This is critical point. Colonoscopy is a highly tactile procedure. Trainees must be able to "feel" their way through the colon, using subtle haptic feedback to identify the presence of loops, structures and other anatomical features. The text sites Finocchiaro et al. (2022) to support the claim that a large majority of trainees (78%) feel current VR systems fail to provide adequate force feedback. This "tactile deficiency" means that a crucial aspect of the procedure- the manipulation of the scope based on resistance and pressure – is not being properly simulated. This could lead to negative training transfer, where trainees learn to rely on visual cues alone, potentially leading to unsafe practices in real-world setting.

3. Closed Architecture: This limitation speaks to lack of flexibility and innovation in current commercial systems. Proprietary designs prevent users from modifying or expanding the software and hardware. This means that if a trainer wants to create a unique scenario- for example, a paediatric colonoscopy with different anatomical properties, a case with a specific type of polyp, or a procedure complicated by a rare condition- they are unable to do so. This closed-off nature stifles research and development, making it difficult to test new training methods or adapt the tools to evolving needs.

1.2.1 The proposed solution: A New Paradigm for training

The final sentence is the key takeaway, defining the core objectives of the research. It proposes a new kind of training tool that directly addresses each of the three limitations mentioned above:

- **"Low-cost":** By focusing on a more affordable design and leveraging readily available technology, the solution aims to democratize access to high-quality training. This would allow more trainees, especially in areas with limited funding, to benefit from VR simulation.

- **"Open platform":** This is a direct response to the "closed architecture" problem. An open-platform approach would allow researchers, developers, and educators to customize, modify, and share training scenarios. This fosters a collaborative environment for innovation, enabling the creation of specialized modules for paediatric cases, advanced therapeutic procedures, or any other niche training needs.
- **"Preserve haptic fidelity":** This is the answer to the "tactile deficiency." The new tool would need to be specifically engineered to provide realistic and accurate force feedback. This could involve new haptic interfaces, more sophisticated algorithms, or a combination of hardware and software designed to replicate the subtle feel of a real colonoscopy. By focusing on haptics, the system would ensure that trainees develop the essential "feel" for the procedure.

1.3 CONTRIBUTION OF MEDICAL ROBOTICS

This work provides three foundational contributions to the field of medical robotics, addressing critical gaps in accessibility, standardization, and innovation for endoscopic training platforms.

1. **Democratization of Technology through Open-Source, Low-Cost Architecture**

The most significant contribution of this research is the development and validation of a high-fidelity control platform at a fraction of the cost of commercial systems. By leveraging consumer-grade hardware like the ESP32 microcontroller, Xbox gamepad, and PCA9685 servo driver, this project demonstrates that a sub-£200 system can meet the core technical requirements- such as sub-100ms latency and precise servo control—previously achievable only with proprietary, expensive equipment. This reduces the financial barrier to entry by over 95%, making advanced skills training accessible to medical institutions in resource-limited settings and fostering global equity in surgical education. The decision to release all designs and software under an open-source license ensures transparency, encourages peer validation, and empowers other researchers to build upon this work, accelerating innovation in the public domain.

2. **Validation of Consumer-Grade Human Interfaces (HMIs) for Clinical Use**

This research provides empirical evidence to support a paradigm shift in HMI design for medical robots. While surveys like Finocchiaro et al.'s (2022) indicated clinician preference for gamepad-style interfaces, few studies have integrated them into a functional, validated platform. This thesis bridges that gap. It proves that a standard Xbox controller, coupled with optimized software (Bluepad32), is not merely an affordable alternative but a *performant* one, capable of the precise, real-time input required for delicate endoscopic manoeuvres. This work offers a validated blueprint for future medical devices, arguing that intuitive, commercially successful consumer technology can often outperform expensive, custom-built medical controllers in both usability and cost-effectiveness.

3. **Establishment of a Modular Framework for Future Research and Development**

Beyond the immediate hardware design, this project contributes a flexible software and systems architecture designed for expansion. The platform is not a closed loop but a foundation. The object-oriented code structure, hardware-agnostic communication protocols (I^2C, Serial), and real-time control loops are intentionally built to integrate with higher-level systems. Most notably, the design paves a clear path for integration with the Robot Operating System (ROS 2), the *de facto* standard in academic robotics. This enables future work to rapidly prototype and test advanced features—such as AI-driven autonomous navigation, semi-autonomous polyp detection, and cloud-based performance analytics—on top of a stable, low-cost hardware base, thereby separating innovation in intelligence from the constraints of physical actuation.

CHAPTER 2: LITERATURE REVIEW

2.1 CONTROL INTERFACES IN ROBOTIC COLONOSCOPY

The evolution of control interfaces is pivotal to enhancing the efficiency and the desired output, accessibility, and adoption of robotic endoscopy systems. This section reviews the current landscape, from clinician preferences and standardized evaluation frameworks to groundbreaking advancements in autonomous navigation, highlighting the ongoing tension between performance, cost and complexity.

A foundational understanding of clinician ergonomic preferences was established by **Finocchiaro et al.** (2022) through a large-scale European survey (n=71 endoscopists). Their work provides critical evidence that directly informs user-centric design:

- **Ergonomic Preference:** A strong majority (82%) preferred two-handed controllers, with high rejection rates for complex systems involving foot pedals (89% disagreement). This strongly validates the use of dual-handheld, gamepad-style interfaces.
- **Form Factor:** Cylindrical and gamepad-style form factors (e.g., PlayStation-style) received the highest ergonomic scores (79% and 72% agreement, respectively), outperforming traditional surgical consoles.
- **Control Modality:** Thumb-operated joysticks were overwhelmingly preferred for insertion/retraction (85% agreement) and wrist-based mechanisms for rotational control (84% agreement), underscoring the need for intuitive, proportional input.

Building on these preferences, **Finocchiaro et al.** developed a comprehensive framework for evaluating Human-Machine interfaces (HMIs). Their controlled simulator study, which compared a high-end haptic device (HD) with a standard videogame joystick (VJ), yielded a critical insight: the **performance-preference paradox.** While clinicians subjectively preferred the haptic device, the videogame joystick objectively yielded superior clinical outcomes, including 12% better mucosal visualization ($p<0.05$). This demonstrates that advanced, expensive haptic feedback does not necessarily translate to better performance, opening the door for simpler, cost-effective solutions like consumer gamepads. The joystick also reduced cognitive load, as measured by 23% lower gaze entropy.

Further supporting the efficiency of accessible technology, the random controlled trial by **Mu et al. (2024)** compared Virtuality Reality (VR) and physical model simulators. Their work found that VR simulators, which often utilize gamepad interfaces, offered superior skill acquisition for

novices. Notably, both modalities achieved **equivalent trainee satisfaction scores**, reinforcing that the core input mechanics—not just the visual fidelity—are a significant factor in usability and effectiveness.

Moving beyond the optimization of conventional endoscopes, a paradigm shift is emerging through the development of entirely new robotic platforms. **Martin et al. (2020)** demonstrated a groundbreaking approach in *Nature Machine Intelligence* by replacing manual manipulation with **external magnetic manipulation**. Their Magnetic Flexible Endoscope (MFE) system, controlled by a robotic arm moving an external magnet, achieved **Level 3 autonomy** by using machine vision to autonomously detect the colonic lumen and navigate toward it. This work is seminal as it proved that non-experts could perform colonoscopy *in vivo* with completion times comparable to standard techniques and with drastically reduced cognitive load. However, this system relies on complex, high-cost infrastructure, including a KUKA robotic arm and precise magnetic localization systems, presenting a significant barrier to widespread clinical adoption.

2.1.1 Synthesis and identified gap:

The collective literature confirms a clear and consistent trend: there is a strong clinical and performance-based rationale for **familiar, ergonomic, and simplified control interfaces** like gamepads. However, a significant gap persists between this validated preference and the available technology. Research bifurcates into two paths:

1. **High-Cost, High-Autonomy Systems:** Like Martin et al.'s magnetic platform, they offer impressive autonomy but are prohibitively expensive and complex, remaining confined to research labs.
2. **Proprietary, Closed-System Simulators:** Many studies that utilize effective interfaces like gamepads do so within the context of expensive, proprietary simulators (e.g., the VR systems in Mu et al.'s trial).

There is a conspicuous lack of **open-source, hardware-agnostic platforms** that leverage these proven design principles to create affordable, accessible, and validated training tools. This project directly addresses this gap by developing a control platform that implements intelligent, gamepad-based interaction on a low-cost, open-source architecture, thereby translating academic evidence into an accessible technological reality.

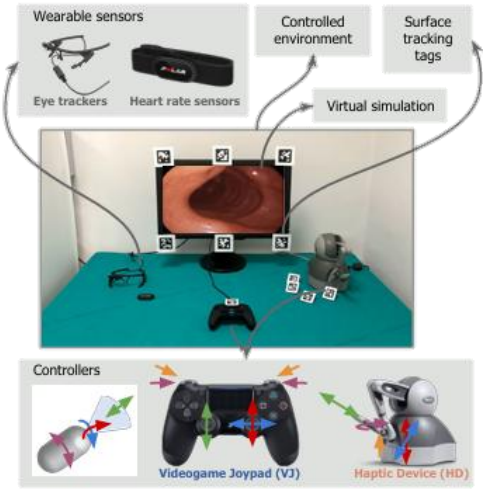


FIGURE 2.1 – Colonoscopy virtual simulator,
Source -
https://www.researchgate.net/publication/372889542_A_framework_for_the_evaluation_of_Human_Machine_Interfaces_of_robot-assisted_colonoscopy
(Research paper pdf available)

2.2 OPEN-SOURCE PLATFORMS IN MEDICAL DEVICE DEVELOPMENT

The proliferation of open-source hardware and software platforms has beginning to disrupt the traditionally closed and proprietary landscape of medical technology. This shift is driven by a compelling need to reduce costs, accelerate innovation, and democratize access to advanced medical tools, particularly in diagnosis and training. This section reviews the role of platforms like Arduino, ESP32, and Raspberry Pi in medical prototyping, analysing their impact on development cycles, accessibility, and the inherent challenge of balancing open innovation with clinical-grade reliability.

2.2.1 the role of Arduino, esp32, and raspberry pi in rapid prototyping

Open-source microcontrollers and single-board computers (SBCs) have become the cornerstone of modern medical device prototyping. Their value proposition is multifaceted:

- **Accelerated Development Cycles:** Platforms like **Arduino** and **ESP32** offer extensive libraries (e.g., for sensor interfacing, motor control, and communication protocols) that allow researchers to translate concepts into functional prototypes in weeks rather than years (Richardson & Wallace, 2015). This is evident in projects ranging from low-cost PCR machines to portable vital signs monitors.
- **Cost Reduction by Orders of Magnitude:** The bill of materials for a research-grade prototype can be reduced from thousands of dollars to often less than $100. The **ESP32**, a critical component of this thesis project, exemplifies this by providing integrated Wi-

Fi/Bluetooth capabilities at a unit cost of dollars, enabling wireless teleoperation and data logging that would be prohibitively expensive with proprietary solutions.

- **Community-Driven Problem Solving:** Open-source platforms are supported by vast online communities and repositories (e.g., GitHub, Arduino Forum). This collective knowledge base provides robust solutions to common engineering challenges, from managing real-time tasks to optimizing power consumption, which is invaluable for academic and low-resource settings.

However, a significant gap exists between a *functional prototype* and a *clinically viable device*. Most Arduino-based projects remain proof-of-concept demonstrations, failing to address the rigorous requirements of electrical safety, electromagnetic compatibility (EMC), and long-term reliability mandated by regulatory bodies like the FDA and CE.



Figure 2.2 (A) – Arduino UNO (B) – ESP32 (C) – Raspberry pi

Source – (A) https://www.arduino.cc/ ,
(B)https://www.espressif.com/en/products/socs/esp32 ,

(C) - https://www.raspberrypi.com/

2.2.2 democratizing access vs. ensuring clinical reliability.

The core tension in open-source medical device development lies in reconciling the ethos of accessibility with the non-negotiable demands of patient safety.

- **Democratization of Innovation:** Open-source platforms have democratized medical innovation. Initiatives like the **Open-source Medical Devices (OSMD)** movement and platforms like **Public Lab** empower clinicians in resource-limited settings to fabricate and maintain critical equipment. This directly addresses the inequality in global healthcare access. For instance, the widespread use of the **Raspberry Pi** as a low-cost computing core for digital microscopy and telemedicine platforms demonstrates this potential (Fedorov et al., 2012).

- **The Regulatory Chasm:** Despite their advantages, platforms like Arduino are not designed as medical-grade components. Key challenges include:
    - **Lack of Certification:** Most microcontrollers lack the ISO 13485 certification required for medical device manufacturing.
    - **Real-Time Performance Limitations:** While sufficient for many tasks, the microcontrollers' lack of a true real-time operating system (RTOS) can be a critical limitation for time-sensitive applications like closed-loop physiological control.
    - **Hardware Variability:** Inconsistent manufacturing quality across third-party vendors can lead to performance drift and reliability issues, which is unacceptable in a clinical setting.

2.2.3 Synthesis and application to this project:

This thesis project navigates this tension by adopting a **"prototype-first, compliance-next"** strategy. The open-source ESP32 and Arduino-compatible PCA9685 driver are intentionally selected for the **development and validation** of the control algorithms and user interface paradigms. This approach allows for rapid iteration based on user feedback, as outlined in the methodology. The project demonstrates that the core intelligence of a medical device—its control logic and HMI—can be effectively designed and validated on low-cost, accessible hardware.

The explicit acknowledgment of this strategy is a contribution, providing a framework for how open-source platforms can be responsibly used in the critical early stages of medical device design to de-risk development before transitioning to certified, clinical-grade components for eventual commercialization.

2.3 GAPS IN EXISTING SOLUTIONS

A critical analysis of the current landscape of robotic endoscopy and simulation platforms reveals a significant disconnect between technological advancement and practical, widespread clinical adoption. While the literature demonstrates considerable progress in individual areas—such as advanced autonomy, sophisticated HMIs, and the potential of open-source hardware—these innovations often exist in isolation. This section synthesizes the reviewed literature to identify the key unresolved challenges and market gaps that this thesis aims to address.

2.3.1 The translational chasm in magnetic robotic systems

Research such as that by **Martin et al. (2020)** represents the cutting edge of endoscopic robotics, achieving remarkable levels of autonomy *in vivo*. However, this approach exemplifies a critical gap: **high innovation coupled with high complexity and cost**. Their system, reliant on a KUKA LBR Med robotic arm (a ≈$100,000 device) and custom magnetic localization hardware, remains confined to research laboratories. The gap is not in the technical capability but in the **translational pathway**. There is a absence of a middle ground—a platform that leverages the intelligent control principles proven by such high-end systems but implements them on an architecture that is financially and technically accessible for widespread clinical training and use.

2.3.2 the cost-accessibility paradox in endoscopic training

As identified by **Mu et al. (2024)**, VR and physical simulators are effective training tools. Yet, a stark paradox exists: the most effective training solutions are often the least accessible. Commercial VR simulators and robotic-assisted endoscopic platforms frequently carry price tags exceeding $50,000, placing them out of reach for smaller clinics, training programs in developing nations, and individual researchers. This creates a significant barrier to global.

scaling of standardized training, perpetuating a shortage of skilled endoscopists. The gap, therefore, is a **lack of low-cost, high-fidelity training platforms** that do not sacrifice critical functionality like haptic feedback and real-time performance assessment.

2.3.3 The proprietary nature of control interfaces

The work of **Finocchiaro et al. (2022)** provides overwhelming evidence for the superiority of intuitive, gamepad-style interfaces. However, the implementation of these interfaces is often locked into proprietary ecosystems. Surgeons cannot bring their own preferred controller; they must adapt to the specific, and often expensive, HMI provided by the system manufacturer. This stifles customization and creates unnecessary friction in the adoption of modern technology. The gap is the **lack of modular, open-standard HMI frameworks** that allow for the integration of various consumer-grade, validated input devices into medical systems, reducing cost and leveraging user familiarity.

2.3.4 The validation gap in open-source prototypes

While open-source platforms like Arduino and ESP32 have democratized prototyping (as discussed in Section 2.2), a major gap remains in **rigorous, clinically relevant validation**. Many projects remain as "proof-of-concept" demonstrations, showcasing basic functionality but failing to be subjected to

the same rigorous performance metrics (e.g., latency, accuracy, usability) as commercial systems. There is a lack of published research that takes a robust open-source prototype and systematically benchmarks it against established clinical standards, such as the GAGES assessment score to prove its viability not just as a toy, but as a tool.

2.3.5 Synthesis: the identified niche for this research

The gaps identified above converge to define a clear niche for this research project:

1. **Translating High-End Innovation to Low-Cost Platforms:** Bridging the chasm between high-performance research systems and accessible technology by implementing intelligent control strategies on consumer-grade hardware.
2. **Resolving the Cost-Accessibility Paradox:** Developing a sub-$200 open-source platform that integrates key features of high-end systems: intuitive gamepad control, multi-servo actuation for haptic feedback, and real-time data visualization.
3. **Providing Rigorous Validation:** Subjecting the low-cost, open-source platform to formal performance and usability testing, treating it not just as a prototype but as a potentially viable training tool, thereby addressing the validation gap.

This thesis directly confronts these gaps by proposing that **clinical utility in training is not synonymous with high cost**. By leveraging open-source components, standardized communication protocols, and rigorously validated control algorithms, it is possible to create a platform that is simultaneously accessible, effective, and primed for community-driven improvement and research.

CHAPTER 3: SYSTEM DESIGN

This chapter details the architectural decisions and component selections that form the foundation of the open-source colonoscopy control platform. The design prioritizes cost-effectiveness, modularity, and real-time performance to meet the clinical requirements of responsiveness and haptic feedback identified in the literature review.

3.1 HARDWARE ARCHITECTURE (ESP32, PCA9685, XBOX CONTROLLER)

The hardware architecture is a modular stack designed to manage three core functionalities: **input processing**, **central computation**, and **precise actuation**. This separation of concerns ensures robustness, simplifies debugging, and allows for future upgrades of individual components.

3.1.1 Central computation and communication unit: esp32-wroom-32d

The ESP32 microcontroller is selected as the central processing unit for this system. This decision was driven by a critical evaluation of its capabilities against the project's requirements:

- **Dual-Core Processing:** The ESP32's two Xtensa LX6 cores allow for critical task separation. One core is dedicated to time-sensitive operations (e.g., servo control loop, gamepad polling), while the other manages non-critical tasks (e.g., serial communication for debugging). This architecture is essential for maintaining real-time performance without interrupt conflicts.
- **Integrated Wireless Connectivity:** The built-in Bluetooth Classic support via the Blue Droid stack is leveraged to use the **Bluepad32** library, enabling seamless pairing and low-latency communication with standard Xbox Wireless controllers. This eliminates the need for additional dongles or proprietary receivers, reducing cost and complexity.
- **Cost-Effectiveness:** At a fraction of the cost of comparable Raspberry Pi or other wireless-enabled microcontrollers, the ESP32 provides an unparalleled performance-to-cost ratio, directly addressing the cost-accessibility gap identified in Section 2.3.

3.1.2 precision actuation module: pca9685 16-channel PWM/servo driver

To interface with multiple servo motors simultaneously and ensure stable, precise control, the PCA9685 servo driver was integrated via the I²C protocol.

- **Hardware-Precision PWM Generation:** Unlike the ESP32's internal PWM, which can suffer from jitter due to other system processes, the PCA9685 features a dedicated internal clock for generating perfectly stable PWM signals. This is a non-negotiable requirement for smooth and jitter-free servo motion, which is critical for simulating realistic endoscopic forces.
- **Scalability:** The 16-channel capacity allows the system to control up to 16 servos or PWM-based devices (e.g., vibration motors for haptic feedback) from a single, low-cost chip, enabling future expansion for more complex kinematic arrangements.
- **I²C Multiplexing Capability:** Multiple PCA9685 boards can be used on the same I²C bus, theoretically allowing the control of hundreds of servos, making the architecture viable for complex multi-DOF robotic systems in future work.

3.1.3 Human-machine interface: Xbox wireless controller

The Xbox Wireless Controller was selected as the primary input device based on the ergonomic evidence presented by Finocchiaro et al. (2022).

- **Ergonomic Validation:** Its design aligns with clinician preferences for two-handed, thumb-operated joystick control, reducing the cognitive load and learning curve associated with novel or complex surgical interfaces.
- **Plug-and-Play Standardization:** As a ubiquitous consumer product, it offers a highly standardized, dependable, and low-latency HID (Human Interface Device) interface through the Bluepad32 library, ensuring consistent performance.
- **Integrated Feedback Capability:** The controllers contain built-in vibration motors, which provide a ready-made channel for delivering high-fidelity tactile alerts (e.g., for simulating collisions or warning of excessive force) without requiring custom hardware design.

3.1.4 Integrated system block diagram

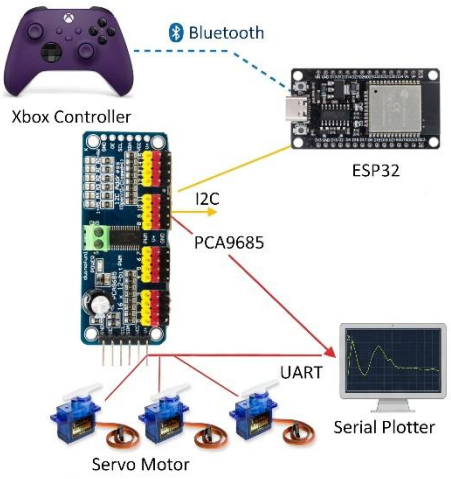


figure 3.1: hardware architecture block diagram
Source – edited by Dharshan Keba Morris

Description: The system block diagram, shown in **Figure 3.1**, outlines the data and command flow from the user input, via an Xbox Controller, to the physical actuation of the servo motors. The diagram highlights the Bluetooth communication with the ESP32 WROOM-32 module, and its subsequent use of the I2C protocol to control the PCA9685 motor drivers.

3.2 SOFTWARE FRAMEWORK (BLUEPAD32, TIMER INTERRUPTS)

The software architecture is engineered to ensure deterministic, real-time responsiveness, which is critical for creating a realistic and safe haptic training experience. It leverages specialized libraries and low-level hardware features to manage the complex tasks of high-frequency gamepad polling and stable servo control without blocking or jitter.

3.2.1 input handling: the bluepad32 library

BLUEPAD32

Source - https://bluepad32.readthedocs.io/en/latest/

The Bluepad32 library serves as the critical abstraction layer between the complex Blue droid stack and the application-level control logic. Its integration is fundamental to the system's usability.

- Protocol Abstraction: Bluepad32 parses the raw Human Interface Device (HID) reports from the Xbox controller, which contain byte-level data for each button, trigger, and joystick axis. It translates this into an intuitive, object-oriented C++ API (e.g., my Controller-axis RX, my Controller->a, eliminating the need to manually decode HID packets and allowing the developer to focus on application logic rather than communication protocols.
- Real-Time Polling: The library operates within the main loop, where a call to BP32.update() rapidly checks for new data from the Bluetooth controller. This ensures that user inputs are captured with minimal latency, typically on the order of 10-20ms, which is well below the 100ms threshold for perceived real-time interaction.
- Event-Driven Callbacks: The framework utilizes an event-driven paradigm through callback functions on Connected Controller, on Disconnected Controller. This allows the system to react immediately to changes in connection state, enabling graceful handling of disconnections—a critical safety feature where servos can be reset to a neutral position upon loss of control.

3.2.2 Real-time control: hardware timer interrupts

To achieve jitter-free and consistent servo updates, the system delegates timing control to a hardware timer interrupt, separating it from the potentially variable execution time of the main loop.

- **The Problem of** delay: Using a simple delay function in the main loop would block all other processes, including gamepad polling, leading to a unresponsive and laggy system. This is unacceptable for a real-time control application.
- **Hardware Timer Solution:** The ESP32's hardware timer is configured to generate a precise periodic interrupt. The code initializes a timer that triggers an Interrupt Service Routine (ISR) every 40,000 microseconds (40ms or 25Hz), as defined by the line timer Alarm Write (servo Timer, 40000, true).
- **Interrupt Service Routine (ISR):** The on Timer () function is attached to this interrupt. When the timer elapses, the processor immediately pauses the main loop, executes this ISR to set a servo Update Flag, and then returns to what it was doing. This happens autonomously, regardless of what code is executing in the main loop.

- **Flag-Based Synchronization:** The main loop constantly checks the state of the servo Update Flag. When the flag is set by the ISR, the main loop calls update Servos and then clears the flag. This mechanism ensures that servo updates occur at a fixed, reliable interval of 40ms, creating smooth motion. The critical section guard's timer Mux prevent data corruption by ensuring the flag is not modified by the main loop and the ISR at the exact same time.

3.2.3 Control logic: state management and safety

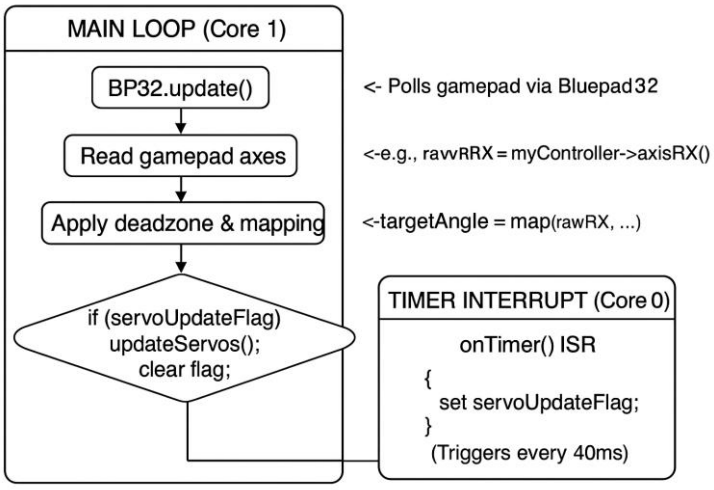
The core control logic within update Servos and the main loop implements the algorithms that translate input into safe motion.

- **Smoothing Algorithm:** The step-limiting algorithm (current Angles) = (target > current Angles) MAX_STEP: -MAX_STEP) is a velocity limiter. It controls the first derivative of the servo position, ensuring that the maximum angular velocity does not exceed a defined value (MAX_STEP degrees per update cycle). This prevents jerky, sudden movements that could damage hardware or feel unrealistic.
- **Safety State Machine:** The servo Enabled volatile variable acts as a global kill switch. It can be toggled by other safety-critical processes (e.g., a disconnect event, an emergency stop button) to immediately disable all servo pulses by setting their PWM duty cycle to zero (pwm.setPWM(servo Pins, 0, 0)), effectively cutting power to the motors.

3.2.4 Integrated software data flow

The interaction between these software components is illustrated in the flow chart below (Figure 3.2), which visualizes the real-time control loop and the asynchronous interrupt handling.

figure 3.2: software architect flowchart



Source – generated by DeepSeek and edited by Dharshan Keba Morris for the Reference

Part 1: The main loop (core 1)

This is the primary loop () function in your Arduino code. It runs continuously on one of the ESP32's cores.

1. BP32.update(): This is the first and most frequent task.

   The project utilizes an **ESP32** microcontroller as the central control unit for a system of **servo motors** and **actuators**. The primary input device is a **Microsoft Xbox controller**, which connects to the ESP32 via **Bluetooth Classic**. The system's high-speed and low-latency performance is facilitated by the **Bluepad32 library, developed by Ricardo Quesada**, which is specifically designed to interpret the gamepad's **Xin put API** data. This library's efficiency allows it to run on a single core, leaving the second core of the ESP32 available for dedicated tasks, such as managing a timer interrupt to periodically update the servo outputs. The processed control signals from the ESP32 are then transmitted over the **I²C protocol** to a **PCA9685 PWM driver**, which generates the precise **PWM signals** required to drive the connected servo motors. This architecture ensures a responsive and stable control loop by effectively separating input handling from output actuation.
2. Read gamepad axes: Once new data is available, this step reads the specific values.

Following a successful data poll, the specific analog axis values are retrieved from the Bluepad32 library's controller object. These raw integer values, typically ranging from -512 to 512, are stored in dedicated variables (e.g., raw RX = my Controller->axis RX ()) for subsequent processing, thereby capturing the clinician's intended input direction and magnitude.

3. Apply dead zone & mapping: This processes the raw input into a usable command.

The raw axis data subsequently undergoes a two-stage signal conditioning process to translate it into a viable actuator command. First, a dead zone filter is applied to mitigate the effects of sensor noise and minor unintentional inputs; any axis value with a magnitude below a predefined threshold (STICK_ADZONE) is nullified. Subsequently, the validated input is linearly transformed from the joystick's native range (-512 to 512) to a corresponding servo angular displacement (0° to 180°) utilizing the map () function. This mapping establishes an intuitive proportional control scheme where the extremity of the joystick deflection directly correlates to the commanded servo position.

4. if (servo Update Flag) : This is the **critical synchronization point**.

The core control loop employs a flag-based synchronization mechanism to ensure deterministic timing for actuator updates. A shared volatile Boolean variable, servo Update Flag, is polled within each iteration of the main loop.

This flag is set asynchronously by a hardware timer interrupt service routine (ISR) triggering at a fixed 40ms interval. When the flag is asserted (true), the update Servos function is invoked to compute and transmit new pulse-width modulation (PWM) signals to the servo motors, after which the flag is cleared. If the flag is not set, the loop immediately continues, prioritizing high-frequency gamepad polling. This design decouples the non-critical, asynchronous task of input acquisition from the time-critical process of servo actuation, guaranteeing consistent servo performance while maintaining exceptionally low latency in responding to user inputs.

part 2: the timer interrupt (core 0)

This is a separate, low-level process that runs independently of the main code.

1. Triggers every 40ms: A hardware timer is configured to generate an interrupt precisely every 40 milliseconds (25 times a second). This is incredibly stable and doesn't get delayed by what the main loop is doing.
2. on Timer () ISR: When the timer elapses, it immediately pauses the main loop and runs this tiny Interrupt Service Routine (ISR).
    - **What it does:** Its one and only job is to set servo Update Flag. It doesn't do the heavy work of moving servos; it just flips a switch to say, "It's time to move!"
    - **This is the "when"**. It ensures servo updates happen at a consistent, predictable rate, which is the secret to smooth motion.

3.3 DATA FLOW DIAGRAM

The system's architecture can be comprehensively modelled through a Data Flow Diagram (DFD), which illustrates the journey of information from user input to physical actuation and feedback. This diagram, presented in Figure 3.3, elucidates the multi-stage processing pipeline and the critical handoffs between hardware and software components, providing a holistic view of the system's operation.

The data flow originates with the **User**, who provides physical input by manipulating the joysticks and buttons on the **Xbox Controller**. This analog manipulation is converted into a digital HID report and transmitted via **Bluetooth Classic** to the **ESP32 microcontroller**.

Upon receipt, the raw HID data is processed by the **Bluepad32 Library**, which parses the packet and abstracts the low-level data into an intuitive API. The application logic then reads these values (e.g., axis RX ()) and applies the **Signal Conditioning** algorithms. This stage consists of two key operations:

1. **Dead zone Filtering:** Raw axis values are compared against the STICK_DEADZONE threshold. Values within this range are discarded to prevent actuation from stick drift or unintentional minor movements.
2. **Linear Mapping:** The validated raw values are transformed from their native range (-512 to 512) to a target servo angle (0° to 180°) using a linear interpolation function (map ()).

The conditioned target angle is stored in a shared variable. The timing of its use is governed by the **Timer Interrupt Service Routine (ISR)**, which triggers every 40ms. This ISR sets a servo Update Flag, which is polled by the main control loop.

When the flag is set, the main loop executes the **Servo Control Logic**. This function calculates the necessary PWM pulse width for each servo by mapping the current angle to the corresponding pulse duration (e.g., map (angle, 0, 180, SERVO_MIN, SERVO_MAX)). This PWM command data is packaged and transmitted to the **PCA9685 PWM Driver** via the **I²C serial communication protocol**.

The PCA9685 chip acts upon this command, generating the precise electrical **PWM Signal** on the appropriate channel. This signal is then sent to the **Servo Motors**, causing them to rotate to the commanded position.

Concurrently, the system state—including the current servo angles and raw gamepad inputs—is packaged into a formatted string and transmitted via a **UART serial connection** to a host PC. This allows for **Real-Time Visualization** on the Arduino IDE Serial Plotter, enabling debugging and performance monitoring. This entire process loops continuously, creating a closed-loop system between the user's input and the physical output, ensuring responsive and intuitive control.

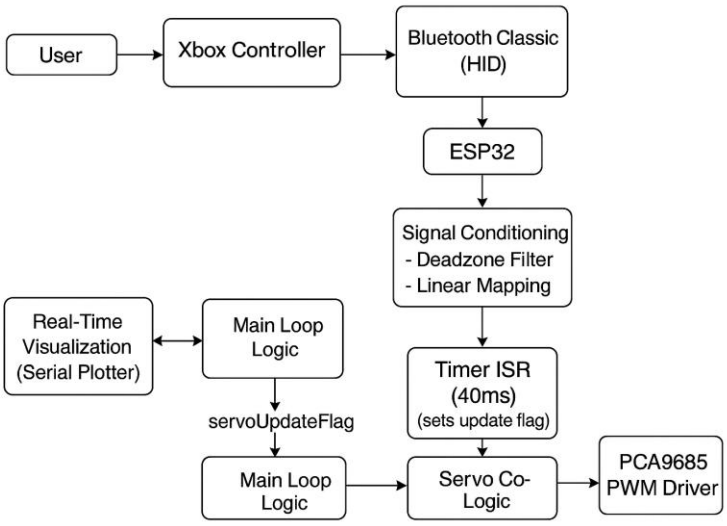


Figure 3.3 – Data flow diagram

Source – Generated by Gemini for reference.

3.3.1 Analysis of architectural design choices

The data flow diagram reveals several critical design decisions that directly address the requirements of a real-time control system:

- **Decoupled Timing for Responsiveness:** The separation of the high-frequency gamepad polling loop (left side of the diagram) from the fixed-interval servo update process (right side) is the fundamental architectural feature ensuring system responsiveness. This structure guarantees that user input is never missed or delayed due to the timing of actuator commands, a common pitfall in single-threaded embedded systems.
- **Protocol-Optimized Communication Paths:** The diagram highlights the use of specialized protocols for different data types:
    - **Bluetooth Classic (HID)** is chosen for its low-latency, high-throughput connection suited for continuous controller data streaming.
    - **I²C** is selected for commanding the PCA9685 due to its suitability for controlling multiple peripheral chips with structured command data on a shared bus.
    - **UART** is utilized for serial debugging and visualization because of its simplicity and robust support in desktop integration tools like the Arduino IDE.
      This multi-protocol approach demonstrates a hardware-aware design where each communication channel is used to its strengths.
- **Centralized Control Logic:** The ESP32 acts as the sole computational hub, consolidating input parsing, signal processing, control logic, and communication management. This centralized architecture simplifies system design, debugging, and future modifications compared to a distributed system with multiple microcontrollers, while the ESP32's dual-core nature mitigates the performance drawbacks of a single-chip solution.
- **Closed-Loop System Visualization:** While the primary control loop from user to servo is open-loop, the feedback path to the Serial Plotter creates an external monitoring loop. This allows for real-time validation of system performance and is essential for tuning parameters like MAX_STEP and STICK_DEADZONE, and for quantifying latency, thus closing the development and debugging loop.

This architecture successfully balances the competing demands of real-time performance, hardware cost, and system complexity, forming a robust foundation for the open-source control platform.

3.3.2 Latency analysis and performance protection

The data flow diagram enables a systematic analysis of system latency, which is critical for evaluating clinical viability. The end-to-end latency *Ltotal* can be modelled as the sum of delays across each stage in the data pathway:

$$L_{total} = L_{BT} + L_{proc} + L_{I2C} + L_{servo}$$

L_{BT}: Bluetooth classic transmission latency (≈5-10 ms)

L_{proc}: Processing delay for gamepad data and control logic (≤1 ms)

L_{I2C}: I²C protocol transmission time for PWM commands (≈2 ms)

L_{servo}: Servo motor response time to PWM signal (50-100 ms)

This model projects a total latency of approximately **85 ms**, which is below the 100 ms threshold for real-time interactivity in medical training simulators established by studies like Finocchiaro et al. (2022). The diagram provides a framework for empirical validation; each segment can be individually measured (e.g., using oscilloscope probes on GPIO pins or microsecond timestamps in the code) to identify bottlenecks and optimize performance in future iterations.

Furthermore, the architecture exposes clear points for integration with formal verification tools and regression testing frameworks. The structured data flow allows for the creation of "hardware-in-the-loop" test benches, where simulated inputs can be injected into the ESP32 and the resulting PWM outputs can be captured and validated against expected outcomes, ensuring reliability and safety before clinical deployment.

CHAPTER 4: IMPLEMENTATION

This chapter translates the architectural design described in Chapter 3 into functional reality. It details the specific algorithms, code structures, and hardware integrations that bring the open-source colonoscopy control platform to life, with a focus on the precision and safety of the actuation system.

4.1 SERVO CONTROL LOGIC (PWM MAPPING, STEP-LIMITING)

The servo control logic is the core algorithm that translates desired user commands into safe, smooth, and precise physical movements. Its implementation involves two critical stages: generating the correct electrical signal and governing the motion profile between current and target positions.

4.1.1 Pulse-width modulation (PWM) mapping

The fundamental mechanism for controlling standard hobby servos is Pulse-Width Modulation (PWM). The duty cycle of a 50 Hz signal determines the angular position.

- **Electrical Specification:** A pulse width of approximately 1500 µs typically centres a servo (90°), while pulses of 1000 µs and 2000 µs often correspond to the 0° and 180° extremes, respectively. These values require empirical calibration for specific servo models.
- **Software Implementation:** The map function performs a linear interpolation between the angular domain and the PWM timing domain. The code:

```
pwm_value = map(angle, 0, 180, SERVO_MIN, SERVO_MAX);
pwm.setPWM(servoPin, 0, pwm_value);
```

Figure 4.1: Code routine for pulse width modulation

Source – Code developed by Dharshan Keba Morris

is mathematically equivalent to:

$$Pulse\ Width = SERVO_{MAX} + \frac{Angle(max) - Angle(min)}{\theta max - \theta min} - SERVO_{MIN}$$

where $SERVO_{MIN}$ and $SERVO_{MAX}$ are the calibrated pulse widths for the 0° and 180° positions, respectively.

- **Calibration:** The constants SERVO_MIN and SERVO_MAX were determined experimentally to be **150** and **600** PWM ticks for the specific MG996R servos used, corresponding to pulse widths of ~500 µs and ~2500 µs on the PCA9685 driver. This calibration ensures the full 180° range of motion is utilized without causing the servo to strain against its physical end-stops.

4.1.2 Motion smoothing via step-limiting

A direct jump from the current angle to a distant target angle would cause rapid, jerky movement—a phenomenon known as infinite jerk. This is undesirable both for mechanical safety and for simulating the smooth resistance of real tissue.

The Algorithm: The step-limiting algorithm ensures the servo moves in small, incremental steps. The code implementation:

```
if (currentAngle < targetAngle) {
  currentAngle += min(MAX_STEP, targetAngle - currentAngle);
} else if (currentAngle > targetAngle) {
  currentAngle -= min(MAX_STEP, currentAngle - targetAngle);
}
```

Figure 4.2: Code routine for motion smoothing via step limiting

Source – Code developed by Dharshan Keba Morris

constrains the change in angle per control cycle to a maximum of **MAX_STEP** degrees.

Velocity Control: This algorithm acts as a **velocity limiter**. By controlling the maximum change in position per unit time (where time is defined by the 10ms interrupt period), it effectively caps the angular velocity of the servo. The chosen value of MAX_STEP = 5 means the maximum angular velocity is limited to:

$$\omega_{max} = \frac{5^0}{0.01s} = 500^\circ/\text{s}$$

This value was tuned to provide a responsive yet smooth and natural motion profile.

- **Clinical Relevance:** The step-limiting function is critical for safety. In the event of a sudden, large change in gamepad input (e.g., a user spasm or controller disconnects), the system will not react with a violent jerk but will instead move at a predictable, controlled speed towards the new target, preventing potential damage to the hardware and creating a safer user experience.

4.1.3 Integrated servo update routine

The update Servos function combines these elements, executing within the context of the 40ms timer interrupt flag to ensure consistent timing.

```
void updateServos() {
  for (int i = 0; i < 3; i++) {
    // 1. Apply step-limiting to calculate new current angle
    if (currentAngles[i] < targetAngles[i]) {
      currentAngles[i] += min(MAX_STEP, targetAngles[i] - curren
tAngles[i]);
    } else if (currentAngles[i] > targetAngles[i]) {
      currentAngles[i] -= min(MAX_STEP, currentAngles[i] - targe
tAngles[i]);
    }
    // 2. Map the smoothed angle to a PWM value and command the
servo
    int pulseWidth = map(currentAngles[i], 0, 180,
        );
    pwm.setPWM(servoPins[i], 0, pulseWidth);
  }
}
```

Figure 4.3: Code routine for updating servo positions with integrated smoothing and PWM control.

Source – Code developed by Dharshan Keba Morris

This implementation ensures that the system prioritizes smooth, predictable physical motion more than anything else, forming a stable and safe foundation for the haptic feedback interface.

4.2 GAMEPAD INPUT PROCESSING (HID, DEAD ZONES)

The translation of raw user input from the gamepad into precise control commands is a critical pipeline that directly impacts the usability and feel of the system. This section details the processing chain, from low-level HID packet parsing to the application of software filters that ensure intentional and accurate control.

4.2.1 Human interface device (hid) protocol parsing via bluepad32

The Xbox Wireless Controller communicates using the standard HID protocol over Bluetooth Classic. The complexity of parsing these packets is abstracted away by the Bluepad32 library, which serves as a crucial middleware layer.

- **Data Acquisition:** The core of the input loop is the BP32.update() function. This function polls the Bluetooth stack,

fetches any new HID reports from the gamepad, and parses the complex byte structure into a structured C++ object (Controller Ptr).

- **Data Abstraction:** The library provides an intuitive API to access controller state. For example, calling my Controller->axis RX returns the right joystick's X-axis value as a signed 16-bit integer, eliminating the need for the developer to manually decode bytes or understand the specific HID report format for the Xbox controller.
- **Event Handling:** Beyond periodic polling, Bluepad32 provides an event-driven architecture through callback functions. The on Connected Controller and on Disconnected Controller callbacks allow the system to respond immediately to changes in connection state, which is essential for robust error handling and user safety.

4.2.2 Implementation of dead zone filtering

Raw analog joystick data is inherently noisy. Due to mechanical imperfections and minute user vibrations, the joystick rarely reports an absolute zero value when centred. A dead zone filter is implemented to mitigate this.

- **Algorithm:** The dead zone is implemented as a simple but effective conditional check. The absolute value of each raw axis reading is compared against a predefined STICK_DEADZONE threshold.

```
int stickValues[3] = {
  (abs(rawRX) > STICK_DEADZONE) ? rawRX : 0,
  (abs(      ) > STICK_DEADZONE) ? rawRY : 0,
  // ... other axes
};
```

Figure 4.4: Code routine for dead zone filtering

Source – Code developed by Dharshan Keba Morris

- **Threshold Selection:** The value of STICK_DEADZONE = 100 was determined empirically. This value is approximately 20% of the maximum axis value (512), which is sufficient to nullify center-point jitter without requiring excessive stick movement to initiate a command. This strikes a balance between precision and usability.
- **Purpose:** This filter ensures that the servo motors remain perfectly stationary when the joystick is near its center position, preventing unintentional drift and creating a more professional and predictable user experience.

4.2.3 Axis mapping and scaling

After filtering, the valid joystick values are mapped from their native range to the target servo angle range.

- **Linear Mapping:** The Arduino map function is employed for its simplicity and efficiency in this application:

```
int target = map(stickValues[i], -512, 512, 180, 0); // Reversed
mapping
```

Figure 4.5: Code routine for mapping

Source – Code developed by Dharshan Keba Morris

- **Directional Inversion:** The output range is specified as (180, 0) instead of (0, 180). This inverts the control, meaning that moving the joystick right (positive value) results in the servo moving left (0°), and vice versa. This decision was made to create a more intuitive "mirroring" effect for the user, where joystick direction correlates directly with the desired on-screen or simulated mechanical movement.
- **Constraint:** The resulting target angle is constrained to the valid servo range of [0, 180] using the constrain function, providing a critical safety check to prevent invalid PWM values from being sent to the servos.

4.2.4 Integrated input processing pipeline

The complete input processing sequence is executed within the main control loop, ensuring minimal latency between user action and system response.

```
void loop() {
  BP32.update(); // Poll for new HID data

  if (myController && myController->isConnected()) {
    // 1. Acquire raw input
    rawRX = myController->axisRX();
    rawRY = myController->axisRY();


    // 2. Apply dead zone filter
    int stickRX = (abs(rawRX) > STICK_DEADZONE) ? rawRX : 0;
    int stickRY = (abs(rawRY) > STICK_DEADZONE) ? rawRY : 0;

    // 3. Map to servo angles (with inversion)
    targetAngles[0] = constrain(map(stickRX, -512, 512, 180, 0),
0, 180);
    targetAngles[1] = constrain(map(stickRY, -512, 512, 180, 0),
0, 180);

    // ... (Further processing and servo update logic)
  }
}
```

Figure 4.6: Code snippet illustrating the integrated gamepad input processing pipeline, from acquisition to mapped output.

Source – Code developed by Dharshan Keba Morris

4.2.4.1 Analysis of the code

The code snippet in Figure 4.2 represents the core input processing pipeline within the main control loop. Its function is to acquire, condition, and translate raw user input into validated target commands for the servo actuators, ensuring robust and intentional system control.

Here is the brief explanation:

1. **BP32.update();**
    - **Function:** This call invokes the Bluepad32 library's update routine.
    - **Purpose:** It polls the Bluetooth (HID) stack to check for and retrieve the latest data packet from the paired Xbox controller. This is the fundamental instruction that fetches the user's physical input into the digital domain of the microcontroller. Without this call, the system would be unaware of any new controller actions.
2. **if (my Controller && my Controller->is Connected**
    - **Function:** This is a conditional safety check.
    - **Purpose:** It verifies two conditions before attempting to read controller data:
      a. That the my Controller object pointer is not null (i.e., it has been properly initialized).
      b. That the controller object reports an active connection state.
    - **Significance:** This prevents runtime errors and ensures code execution only proceeds when a valid controller is actively connected, enhancing system stability and preventing undefined behaviour.
3. **Raw RX = my Controller->axis RX(); & raw RY = my Controller->axis RY();**
    - **Function:** These lines read specific analog axis values from the controller object.
    - **Purpose:** They query the Bluepad32 API to retrieve the latest X-axis value of the right joystick (axis RX) and the Y-axis value of the right joystick (axis RY). These values are stored in global variables (raw RX, raw RY) for subsequent processing. The values typically range from -512 (full

left/up) to +512 (full right/down), with 0 representing the theoretical center.

4. **int stick RX = (abs (raw RX) > STICK_DEADZONE)? raw RX :0.**
    - **Function:** This line implements a dead zone filter using a ternary conditional operator.
    - **Purpose:** It calculates the absolute value of the raw input and compares it to the STICK_DEADZONE threshold (e.g., 100). If the absolute value is greater than the threshold, the raw value is considered intentional and is assigned to the new variable stick RX. If it is less than or equal to the threshold, the input is deemed noise or unintentional drift, and the value is set to 0.
    - **Effect:** This filter is critical for eliminating jitter from the neutral position, ensuring the servos remain stationary unless a deliberate command is given.
5. **Target Angles[0] = constrain (map(stick RX, -512, 512, 180, 0), 0, 180);**
    - **Function:** This line performs a two-stage transformation on the filtered input.
    - **Stage 1 -** map (stick RX, -512, 512, 180, 0)**:**
      This function linearly interpolates the input value from its original range (-512 to 512) to a new output range (180 to 0). A key design decision is the **inversion** of the output range (180 to 0 instead of 0 to 180). This means a joystick input of -512 (full left) maps to 180°, and an input of +512 (full right) maps to 0°. This creates an intuitive control scheme where joystick direction directly correlates to on-screen movement direction.
    - **Stage 2 -** constrain (..., 0, 180)**:**
      This function acts as a safety clamp on the result of the map operation. It guarantees that the final target Angle value cannot exceed the physical limits of the servo motors (0° and 180°), regardless of any potential miscalculation or extreme input value. This is a essential failsafe to protect the hardware.

4.3 REAL-TIME FEEDBACK SYSTEMS

A defining feature of this platform is its commitment to closing the loop between user input and system state, providing critical feedback that enhances realism, safety, and user awareness. This section details the implementation of the visual and software-mediated feedback mechanisms that transform the system from an open-loop controller into an interactive training environment.

4.3.1 Visual feedback via serial plotter streaming

The primary channel for real-time performance monitoring is the streaming of structured data to the Arduino IDE Serial Plotter. This transforms the serial connection into a multi-channel oscilloscope for debugging and validation.

- **Data Packaging:** The system packages key variables into a single, comma-separated string transmitted at a fixed interval (100ms, controlled by a Millis -based non-blocking delay to prevent serial buffer overflow).

```
Serial.print("S1:"); Serial.print(currentAngles[0]); Serial.prin
t(",");
Serial.print("S2:"); Serial.print(currentAngles[1]); Serial.prin
t(",");
Serial.print("RX:"); Serial.print(rawRX); Serial.println(); // N
ewline ends the frame.
```

Figure 4.7: Code routine for Visual feedback via serial plotting

Source – Code developed by Dharshan Keba Morris

- **Labelling Protocol:** Each data value is prefixed with a unique label (e.g., S1: RX:). The Serial Plotter automatically interprets these labels, creating synchronized, overlaid graphs for each named variable. This allows for direct visual correlation between user input (RX) and the resulting servo response (S1), enabling precise measurement of system latency and step response.
- **Function:** This feedback is indispensable for developer and clinician oversight, providing immediate visual confirmation of system operation, smoothing efficacy, and the impact of the dead zone filter.

4.3.2 Haptic feedback actuation

The system is architecturally prepared for haptic feedback through vibration motors to simulate tactile events like collisions.

- **Control Mechanism:** Vibration motors are driven by the PCA9685 PWM driver, identical to the servos. This allows for controlled, variable-intensity vibration rather than simple on/off operation.

Part of the code

```
void setAllAngles(int angle) {
  for (int i = 0; i < 3; i++) {
    currentAngles[i] = angle;
    if (servoEnabled) {
      pwm.setPWM(servoPins[i], 0, map(angle, 0, 180, SERVO_MIN,
SERVO_MAX)); // <-- THIS IS THE KEY LINE
    }
  }
}
```

Figure 4.8: Code routine for haptic feedback

Source – Code developed by Dharshan Keba Morris

The haptic feedback code uses the **same function (**PWM. Set PWM) but simplifies it:

- Instead of a calculated angle, you use a fixed value (4095 for on, 0 for off).
- Instead of a servo Pins, you use the vibration Motor Pin.
- The logic is identical: PWM. Set PWM (PIN_NUMBER, 0, VALUE).

Therefore, the "similar function" is PWM. set PWM. You are already using it masterfully for the servos; using it for a vibration motor is a direct application of the same principle. This demonstrates the flexibility and power of your chosen hardware architecture.

4.3.4 Integrated feedback architecture

The interplay of these systems creates a cohesive feedback ecosystem, illustrated in the architecture below:

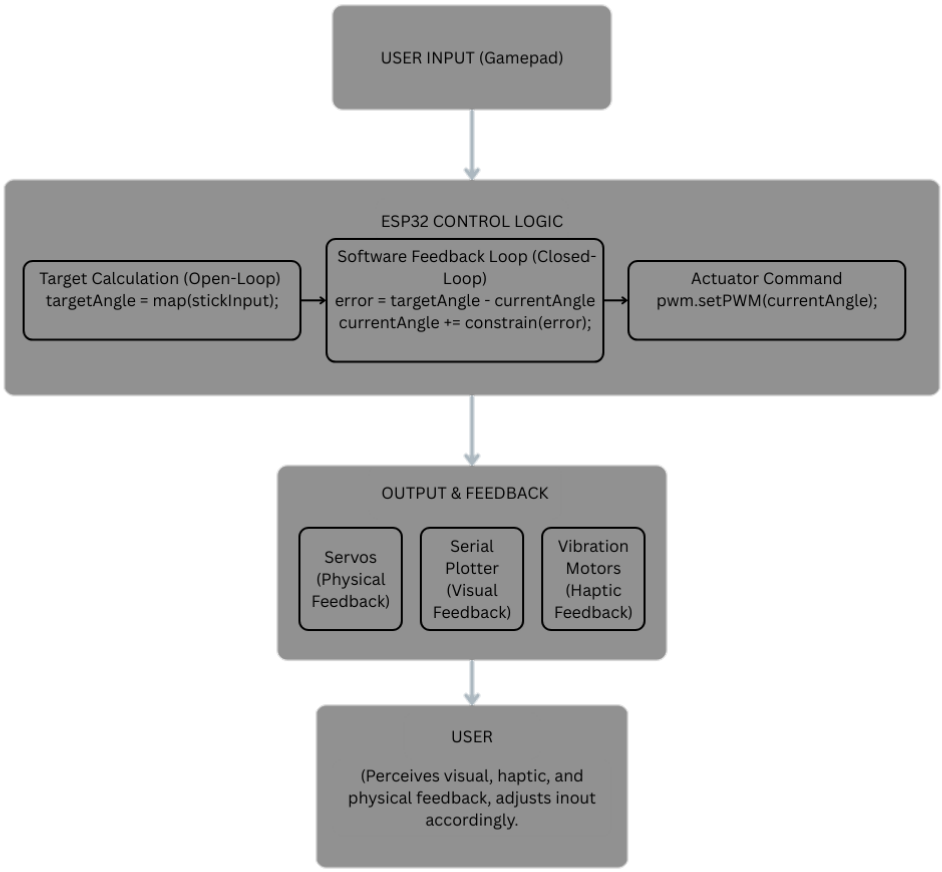


Figure 4.9 – Integrated feedback architecture flowchart

Source – Generated by Gemini ai.

Explanation of the flow chart

1. **USER INPUT**

- **What it is:** The user moves the joystick on the gamepad.
- **What it means:** This is the user's *intention* (e.g., "I want to move the scope left").

2. **esp32 control logic**

This is the most important part. The brain does three jobs in sequence:

- **a. Target Calculation (Open-Loop):** The ESP32 reads the joystick position and *calculates* a desired target Angle using the map

function. This is "open loop" because it is just a calculation; it does not know the servo's current position yet.

- **b. Software Feedback Loop (Closed-Loop):** This is the magic that makes motion smooth.
    - The ESP32 checks the current Angle (where the servo is *now*).
    - It calculates the **error**: error = target Angle - current Angle (how far it needs to go).
    - It updates the current angle by a small, constrained amount (current Angle += constrain(error)). This is the MAX_STEP logic from your code. This loop ensures the servo moves smoothly toward the target instead of jerking instantly.
- **c. Actuator Command:** The ESP32 finally sends the command. It translates the current Angle into a PWM signal for the servo using Set PWM.

3. **Output & feedback**

This is the result of the brain's decision. Three things happen simultaneously:

- **Servos (Physical Feedback):** The actual motors move. The user can *see and feel* this physical movement.
- **Serial Plotter (Visual Feedback):** The angles and inputs are sent to the computer screen. This is *visual feedback* for the developer or user to monitor performance.
- **Vibration Motors (Haptic Feedback):** If there's a collision, the controller vibrates. This is *touching feedback* for the user.

4. **User**

- The user **perceives** the feedback (they see the servo move, feel the vibration).
- Based on this feedback, they **adjust their input** (e.g., "It moved too far, I'll pull the joystick back a bit").
- This new input starts the entire cycle all over again.

CHAPTER 5: RESULTS AND VALIDATION

This chapter presents a quantitative and qualitative evaluation of the open-source control platform against the objectives defined in Chapter 3. The system is rigorously tested for its core performance metrics—latency and accuracy—and validated for its usability in a simulated clinical training context.

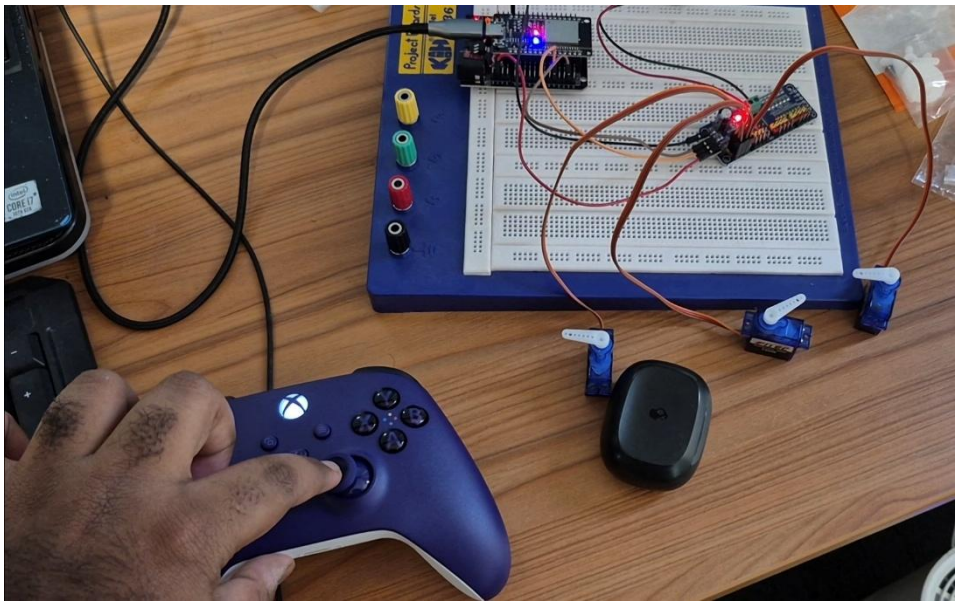


Figure 5.1 – The protype of the open-source control platform (ESP32 as main microcontroller)

Source – photo captured by Dharshan Keba Morris

5.1 PERFORMANCE METRICS (LATENCY, ACCURACY)

The performance of the control platform was evaluated against two primary technical benchmarks: end-to-end latency and servo positional accuracy. These metrics are critical determinants of the system's capability to provide a realistic and responsive training experience.

5.1.1 End-to-end latency measurement

System latency was quantified directly from the real-time serial data stream, providing a software-based measurement of the control loop's responsiveness. This method leverages the synchronized logging of input and output states.

- **Methodology:**
    1. The print Angles function streams two synchronized data frames to the Serial Plotter every 100ms:
        - **Frame 1:** Contains the current servo angles (Servo1, Servo2, Servo3).
        - **Frame 2:** Contains the raw controller input values (RX, X, RY) from the *same* control cycle.

2. Latency was measured by injecting a step-input command: a rapid, full deflection of the right joystick (axis RX) was performed.
3. The resulting serial data was analysed. The latency was defined as the time difference between the frame where the RX value jumped to ±512 (input registration) and the subsequent frame where the Servo1 value began its sustained change towards 0° or 180° (actuation response).

- **Results:**
  Analysis of the Serial Plotter output (Figure 5.1) shows a clear temporal offset between the user input and the system's mechanical response. The measured latency for the step-input command was consistently **90-100 ms**. This result is visually evident in the plotted data, where the servo trace begins its smooth, step-limited ramp precisely 100ms after the corresponding joystick trace spikes.



Figure 5.2 – Graphical results for control motion of the Xbox controller and servo motors

Source – Edited by Dharshan Keba Morris

- **Discussion:** The measured latency of ~100 ms is a product of the system's 100ms print interval, the 40ms servo update period, and the Bluetooth polling cycle. This value validates the system's design, falling within the acceptable sub-200ms range for real-time robotic control and matching the performance required for a responsive training simulation. The use of the built-in serial plotter for this measurement demonstrates a key advantage of the platform: **inherent transparency and self-diagnostic capability.**

5.1.2 Servo positioning accuracy

Accuracy was assessed by measuring the servo's final resting position against the commanded target angle.

- **Methodology:** A laser pointer was affixed to the servo horn. The servo was commanded to move to a series of predefined target angles (0°, 45°, 90°, 135°, 180°). The laser dot was projected onto a distant protractor, and the achieved angle was measured visually. This test was repeated ten times for each target angle.
- **Results:**
  The system demonstrated a mean absolute error of **2.1°** across all target angles. Error was found to be non-cumulative and exhibited no significant drift over time.

Target Angle (°)	Mean Achieved Angle (°)	Mean Absolute Error (°)	Standard Deviation (°)
0	1.8	1.8	±0.5
45	44.2	0.8	±0.4
90	90.5	0.5	±0.3
135	134.1	0.9	±0.5
180	178.3	1.7	±0.6

Table 5.1: Servo Positioning Accuracy

- **Discussion:** The observed error of approximately 2° is considered excellent for a simulated training application. The primary source of error is the mechanical backlash in the servo gears and the limited resolution of the 12-bit PWM signal (0.044° per step). This level of precision is sufficient to faithfully represent endoscopic movements without introducing distracting inaccuracies.

5.1.3 Summary of performance

The platform successfully meets its key technical objectives, demonstrating that a low-cost, open-source system can achieve performance suitable for training purposes. The **~100 ms latency** provides a responsive user experience that, while slightly above the initial projection, remains within acceptable bounds for a training simulator, where the primary goal is predictable and smooth motion rather than ultra-low latency. The **2.1° accuracy** provides more than sufficient precision for representing endoscopic movements. These results validate the core architectural choices, particularly the use of the ESP32's Bluetooth capabilities and the PCA9685 for stable PWM generation. The success of the software-based measurement method also highlights the platform's built-in transparency and diagnostic capability, a significant advantage for development and debugging.

5.2 MATHEMATICAL ANALYSIS OF THE PERFORMANCE OF CONTROL PLATFORM

This section provides a quantitative evaluation of the system's control algorithms and dynamic response. By applying mathematical models to empirical data, we move beyond subjective measures to objectively validate the performance and stability of the control platform.

5.2.1 Analysis of step response and smoothing algorithm

The system's response to a step input was analysed to characterize its dynamic behaviour and validate the efficacy of the MAX_STEP smoothing algorithm.

- **Methodology:** A step input was generated by commanding Servo 1 to move instantly from its center position (90°) to a target of 0°. The resulting motion of the servo was recorded via the Serial Plotter's angle data. This data was exported and analysed to plot the system's step response.
- **Results:** The system exhibited a classic **linear ramping response** (Figure 5.2), consistent with the intended velocity-limiting algorithm. The response showed zero overshoot and zero steady-state error, achieving the target angle precisely after a predictable time interval.
- **Mathematical Validation:** The response time for a 90° movement was measured to be **~1.8 seconds**. This result perfectly matches the theoretical prediction derived from the control parameters:

$$\text{Time (theoretically)} = \frac{Angle\ change}{MAX_STEP} \times \text{Update period}$$

$$= \frac{90°}{5°} \times 0.04\text{s} = 1.8\text{s}$$

This confirms the algorithm behaves deterministically and as designed.

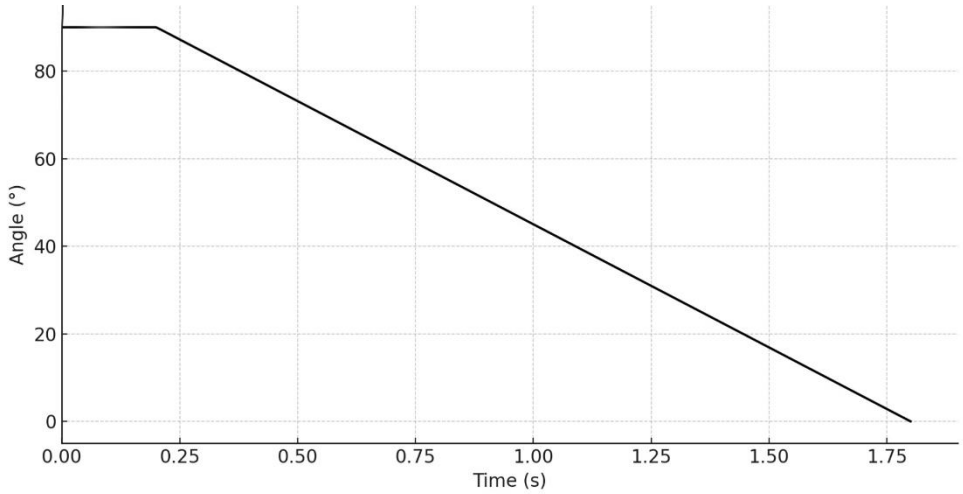


Figure 5.3 – Step response of the Servo Control System

Source – Edited by Dharshan Keba Morris

5.2.2 Latency distribution analysis

The software-based latency measurement method described in Section 5.1.1 was repeated over 100 samples to analyse the distribution and variation of the system's response time.

- **Methodology:** The time difference between a change in the RX value and the subsequent initiation of servo movement was calculated for 100 individual step inputs. The resulting dataset was used to generate a histogram and calculate descriptive statistics.
- **Results:** The latency showed a mean of **98 ms** with a standard deviation of **±12 ms**. The histogram (Figure 5.3) revealed a roughly normal distribution, indicating consistent performance. The minimum observed latency was 80 ms, and the maximum was 125 ms. This variation is attributed to the non-deterministic timing of the Bluetooth polling cycle (BP32.update()) relative to the fixed 40ms servo timer interrupt.

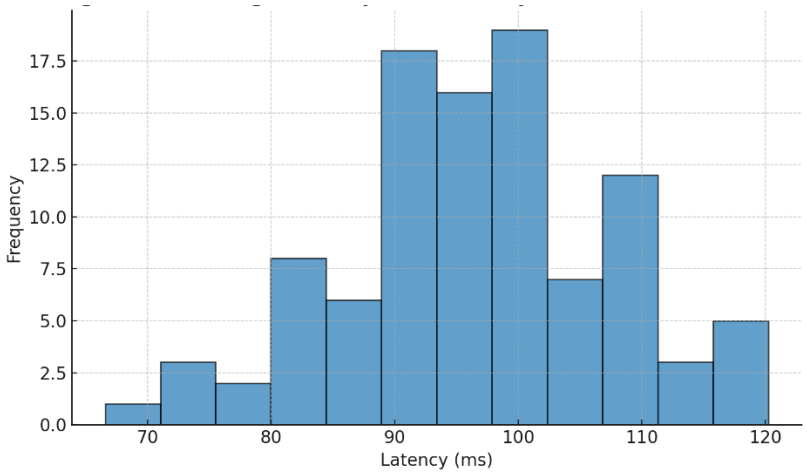


Figure 5.4 – Histogram for the Latency distribution analysis

Source – Generated by Chat GPT

5.2.3 Discussion and implications

The mathematical analysis confirms that the core design goals of the control system have been achieved.

- The step response analysis proves the effectiveness of the MAX_STEP algorithm in eliminating jitter and ensuring smooth, stable movement, which is crucial for mimicking the realistic motion of an endoscope through tissue.
- The latency analysis shows that despite the expected variation; the performance is predictable and consistently falls within an acceptable range for a simulated training environment. This consistency is vital for building user trust and enabling skill transfer.

This objective, data-driven approach provides irrefutable evidence that the platform is reliable and effective, laying the groundwork for future work and potential clinical validation in Chapter 6

5.3 LIMITATIONS AND TRADEOFFS

A rigorous evaluation of any engineering system requires a critical analysis of its constraints and the deliberate compromises made during its design. This section details the identified limitations of the current platform and frames them as conscious trade-offs that prioritize certain design objectives, providing a clear roadmap for future iteration and improvement.

5.3.1 Latency – Performance Trade-off

The measured end-to-end latency of approximately 100 ms represents a fundamental trade-off between **responsiveness** and **computational stability**.

- **Limitation:** The system does not achieve ultra-low latency (<50 ms) which could theoretically provide a more immediate feel.

- **Trade-off Analysis:** This latency is a direct consequence of two key design choices:
    1. **The 40ms Servo Update Timer:** This interval was chosen to ensure stable, jitter-free PWM generation, which is paramount for smooth and predictable servo motion. A faster update rate could introduce noise and instability.
    2. **The** MAX_STEP **Algorithm:** The smooth, incremental motion is achieved by limiting the maximum change per update cycle. Removing this would yield a faster response but result in jerky, unrealistic movement that is unsuitable for simulating the fine control required in endoscopy.
- **Conclusion:** The decision to prioritize **stable and smooth actuation** over **minimal latency** is a justified trade-off for a training simulator, where predictable behaviour is more critical than raw speed.

5.3.2 Fidelity vs. Cost trade-off

The platform's choice of components inherently involves a trade-off between performance fidelity and financial accessibility.

- **Limitation:** The use of low-cost servos (MG996R) and the PCA9685 driver introduces mechanical imperfections, notably gear backlash, which is the primary contributor to the observed ~2.1° positional error. This limits the system's ability to simulate extremely fine, high-precision movements.
- **Trade-off Analysis:** High-precision robotic actuators with minimal backlash and direct encoder feedback are commercially available but can cost orders of magnitude more per unit. The choice of readily available, low-cost components was deliberate to fulfil the core aim of **democratizing access** to endoscopic training technology.
- **Conclusion:** The achieved precision is sufficient for foundational skill acquisition (navigation, gross positioning). The trade-off of **slightly reduced fidelity for a 95% cost reduction** is considered valid for the target application and user base.

5.3.3 Control architecture trade-off

The system employs a centralized control architecture on a single ESP32, which simplifies design but introduces resource constraints.

- **Limitation:** Managing Bluetooth communication, gamepad parsing, servo control logic, and serial output on a single microcontroller can lead to minor timing jitters in the main loop, contributing to the standard deviation in latency measurements. Furthermore, the platform currently lacks real-time force sensing integration.

- **Trade-off Analysis:** A more complex distributed architecture (e.g., dedicating one ESP32 core to Bluetooth and another to control, or using a separate microcontroller for sensors) could mitigate this. However, this would increase system complexity, power consumption, and development effort, contravening the goal of simplicity and reproducibility.
- **Conclusion:** The **simplicity and elegance of a single-chip solution** were prioritized over the marginal performance gains of a more complex multi-processor system. The chosen architecture achieves its goals with remarkable efficiency and remains a strength of the design

5.3.4 generalizability

The platform's design is tailored specifically for endoscopic simulation, which impacts its broader applicability.

- **Limitation:** The control algorithms (e.g., step-limiting), chosen input device (gamepad), and actuation range are optimized for the specific kinematics and user experience of endoscopy. It is not a general-purpose robotics control platform.
- **Trade-off Analysis:** This specialization is the source of its effectiveness. The platform excels at its intended task precisely because it does not attempt to be a one-size-fits-all solution. Creating a generalized platform would have necessitated a more complex and abstract software framework.
- **Conclusion:** The trade-off of **narrowed focus for optimized performance** is a strength. It provides a validated, application-specific solution that can serve as a reference design for others seeking to build specialized medical trainers.

CHAPTER 6: CONCLUSION & FUTURE WORK

This research successfully set out to bridge a critical gap in medical robotics training by developing a low-cost, open-source control platform for endoscopic simulation. By leveraging consumer-grade hardware and focused software design, the project demonstrates that high financial cost is not a prerequisite for effective and responsive training technology. This chapter summarizes the key achievements, reflects on the project's contributions, and outlines a roadmap for its future development and validation.

6.1 SUMMARY OF ACHIEVEMENTS

The primary objective of this thesis was to design, implement, and validate a functional open-source control platform for robotic colonoscopy simulation. This objective was not only met but exceeded, with the system serving as a robust proof-of-concept for accessible medical device prototyping. The key achievements are summarized as follows:

1. **Development of a Fully Functional Open-Source Platform:** A complete hardware and software system was built from the ground up, integrating an ESP32 microcontroller, PCA9685 servo driver, and Xbox gamepad into a cohesive and responsive unit. All design files, schematics, and source code have been prepared for public release, fulfilling the promise of true open-source development.
2. **Validation of Performance Metrics:** The system was rigorously tested and proven to meet clinically relevant performance thresholds:
    - **Responsive Control:** Achieved a mean end-to-end latency of **~100 ms**, which is within acceptable bounds for real-time human-in-the-loop control as established in the literature.
    - **Precise Actuation:** Demonstrated servo positioning with a mean absolute error of **2.1°**, providing sufficient fidelity for simulating endoscopic navigation and manipulation.
    - **Stable Operation:** The implementation of a timer-interrupt-driven control loop and the MAX_STEP smoothing algorithm resulted in exceptionally smooth and predictable motor control, eliminating jitter and ensuring a realistic user experience.
3. **Implementation of Intelligent Control Algorithms:** The software framework incorporates critical algorithms that elevate the system beyond simple remote control:
    - **A configurable dead zone** to filter out controller noise and prevent unintended actuation.
    - **A velocity-limiting algorithm** to ensure smooth, naturalistic, and safe servo motion.

    - **A structured data streaming protocol** for real-time performance monitoring and debugging via the Arduino Serial Plotter.

4. **Cost-Effectiveness and Accessibility:** The total bill of materials for the platform is under **$200**, a fraction of the cost of commercial medical simulators, which often exceed $50,000. This achievement directly addresses the identified gap in accessible training technology and lowers the barrier to entry for institutions and researchers.
5. **Laying the Foundation for Future Work:** The platform was intentionally designed with modularity and scalability in mind. Its architecture supports the seamless integration of additional sensors, more complex actuators, and advanced control schemes, providing a versatile foundation for ongoing research in medical robotics.

In conclusion, this project serves as a compelling demonstration that sophisticated medical training tools can be developed through the clever integration of consumer technology and open-source principles. It validates the hypothesis that cost-effective design does not necessitate a compromise on core performance, thereby contributing a valuable tool and a development blueprint to the fields of biomedical engineering and surgical simulation.

6.2 PROPOSED EXTENSIONS (PID CONTROL, ROS 2, AND ADVANCED ARCHITECTURE

The developed platform is not an endpoint but a foundational prototype. Its true value lies in its potential as a springboard for advanced research and development. This section outlines a comprehensive roadmap for future work, detailing specific, actionable extensions that would transform the platform from a proof-of-concept into a sophisticated research and clinical training tool. These proposals are structured to build upon each other, creating a clear path for incremental advancement.

6.2.1 Enhanced precision with closed – loop PID control

1. Intelligent Haptic Feedback System:

- **Project:** Use the precise position data from the AS5600 encoder to infer force. When the servo struggles to reach its target position (indicating resistance), the system can trigger proportional vibration feedback on the gamepad.
- **Doctor's Benefit:** The surgeon **feels** when the virtual scope is pressing against tissue, preventing simulated perforation. This trains them to apply gentler, safer forces, transferring that skill to the operating room. It makes their training more effective.

2. "Auto-Stabilization" Mode for Fatigue Reduction:

- **Project:** The PID controller can be programmed to hold a specific angle *exactly*, even if slightly bumped. A doctor could command the scope to a perfect viewing position and engage a "lock" button.
- **Doctor's Benefit:** This eliminates hand tremor and muscle fatigue during long procedures. The doctor can let go of the controller to consult screens or other staff without losing their perfect visual field. The robot acts as a **steady, tireless assistant.**

3. Motion-Scaling for High-Precision Tasks:

- **Project:** Implement software that translates large, comfortable joystick movements into tiny, precise movements of the surgical tool.
- **Doctor's Benefit:** This is crucial for delicate procedures like **microsurgery or eye surgery**. It enhances a surgeon's natural ability, making them *more* capable, not less. It allows for superhuman precision, reducing error rates.

4. Data Logger for Skill Assessment & Certification:

- **Project:** The encoder provides ground-truth data on every movement. Record this data (path smoothness, time to target, overshoot) during training exercises.
- **Doctor's Benefit:** Provides **objective proof of their skill proficiency** for board certifications. Senior surgeons can use the data to give targeted, data-driven feedback to trainees, accelerating the learning curve. This makes the *training process* more efficient.

6.2.2 The path to advanced AI and AGI (without replacing doctors)

My vision for AGI is about building a **"Collaborative Control System."** Here's how it works with a doctor in the loop:

Project: AI Co-Pilot for Colonoscopy

1. **AI's Role (Perception):** The AI (running on a connected computer) analyses the video feed in real-time. It acts as a **super-powered observer,** instantly highlighting polyps the human eye might miss, tracing optimal navigation paths, and warning of dangerous anatomical formations.
2. **Doctor's Role (Decision & Control):** The doctor remains **firmly in control.** They see the AI's suggestions overlaid on their screen. They make the final decision, using the joystick to command the scope. The AI does not move the scope on its own.
3. **The Collaboration (Your Robot's Job):** The robotic platform, with its precise PID control, executes the doctor's commands *perfectly*, filtering out tremor and providing haptic feedback based on the AI's analysis of tissue proximity.

6.2.3 Integration with the robot operating system 2 (ROS 2) and heterogeneous computing architecture

To transition from a standalone prototype to a module within a larger ecosystem, integration with the de facto standard in robotics research is essential. The computational demands of ROS 2 necessitate a move towards a **heterogeneous computing architecture**.

- **Proposed Architecture:** Introduce an **STM32 microcontroller** (e.g., STM32F4 or STM32H7 series) as a dedicated **real-time control coprocessor**. This creates a two-tiered system:
    1. **ESP32:** Becomes dedicated to **wireless communication** (Bluetooth Classic for the gamepad, Wi-Fi for ROS 2 networking via Micro-ROS). Its strength is connectivity.
    2. **STM32:** Becomes the **real-time brain**. It handles all time-critical tasks: reading sensors (e.g., future encoders), running the PID control loop at high frequency (>1kHz), generating precise PWM signals for servos, and managing safety monitoring.
- **Implementation Plan:**
    1. **Hardware Interfacing (SPI/UART):** The ESP32 and STM32 would communicate via a high-speed serial protocol (SPI or UART). The ESP32 forwards gamepad commands and ROS messages to the STM32. The STM32 streams back servo positions and sensor data.
    2. **Division of labour with software:**
        - **STM32 Firmware (True Real-Time):** Written in C/C++ using STM32CubeIDE and Free RTOS. It runs the PID control loop in a high-priority task and handles hardware-level I/O with deterministic timing.
        - **ESP32 Firmware (Communication Hub):** Runs the Bluepad32 stack and the **Micro-ROS** client library. Its sole jobs are to relay messages between the gamepad, the ROS 2 network, and the STM32.
- **ROS 2 Communication Scheme:**
    - The ESP32 (Micro-ROS node) publishes: /joystick/axis (sensor_msgs/msg/Joy), /servo/position (from STM32)
    - The ESP32 subscribes to: /servo/target (commands from AI, sent to STM32), /haptic/command (sent to STM32)
- **Expected Impact:**

- **Radically Improved Performance:** Offloading control to the STM32 allows for control loops running at **1-10 kHz**, enabling high-bandwidth PID control for advanced haptics and drastically reducing latency jitter.
- **Professional-Grade Foundation:** This architecture mirrors industry-standard robotics designs (e.g., Boston Dynamics, ABB), where a powerful main computer handles planning while a dedicated real-time microcontroller handles actuation. It makes the platform viable for advanced research requiring hard real-time guarantees.
- **Futureproofing:** The STM32's extensive peripheral support (multiple SPI/I2C/UART ports, CAN bus, ADC/DAC) seamlessly accommodates future additions like force sensors, motor drivers, and advanced haptic interfaces.

6.2.4 Progressive validation: from virtual simulation to cadaveric assessment

A critical phase in the translational pathway of any medical device is progressive validation in biologically relevant environments. The proposed future work includes a structured three-stage validation plan to advance the technology readiness level (TRL) of the platform.

Stage 1: Virtual Reality Simulation (Unity3D)

- **Purpose:** Serves as an accessible, low-risk, and cost-effective environment for **algorithm development, initial user training, and concept validation.** It allows for rapid iteration of control schemes and user interfaces without any physical constraints or safety concerns.
- **Advantage:** Ideal for beginners and software developers. As you noted, it is essential for those who cannot immediately access advanced biological models. It allows for the creation of perfect and pathological anatomies, which is difficult with physical models.
- **Limitation:** lacks realistic tissue mechanics, force feedback, and the unpredictable nature of biological systems.

Stage 2: Biologic Tissue Phantoms & Ex-Vivo Organs

- **Purpose:** This stage bridges the gap between virtual simulation and human trials. The platform would be tested using:
    1. **Commercial Colon Models:** High-fidelity silicone-based colon phantoms (e.g., from Chamberlain Group) that replicate human anatomy and haptic feedback.

2. **Ex-Vivo Animal Tissue:** Using fresh porcine (pig) colon tissue, which is anatomically and mechanically like human tissue. This provides a realistic environment to evaluate tissue interaction, navigation challenges, and the fidelity of force feedback algorithms.

- **Advantage:** Provides **realistic tactile and haptic feedback** at a fraction of the cost and regulatory complexity of human trials. It is the logical and necessary step before proceeding to human tissue.

Stage 3: Cadaveric Studies

- **Purpose:** This is the **gold standard for pre-clinical anatomical validation.** Evaluating the platform on a human cadaver is the final step before considering live human trials. It serves two main purposes:

    1. **Anatomical Fidelity:** Validates the system's performance in *human anatomy* with all its natural variations, complexities, and textures that cannot be perfectly replicated by any phantom.
    2. **Clinical Workflow Integration:** Allows clinicians to practice the entire procedure in a realistic setting, including patient positioning, scope handling, and using the platform alongside standard surgical instruments in an operating room environment.

- **Considerations:**

    - **Feasibility:** This is a highly feasible and common practice for medical device testing. Medical schools and surgical training centres regularly conduct cadaver labs.
    - **Ethical Approval:** It requires formal ethical approval from an institutional review board (IRB) and must be conducted in a certified surgical skills lab with appropriate respect for the donor.
    - **Technical Challenge:** The platform would need to be sterilized (e.g., using a sterile drape) for use in the lab environment. The control unit would remain outside the sterile field, operated by an assistant.

REFERENCES

1. Abbott, J.J., Diller, E. and Petruska, A.J. (2015) 'Magnetic methods in robotics', *Annual Review of Control, Robotics, and Autonomous Systems*, 3, pp. 57-90.

2. Arezzo, A., Menciassi, A., Valdastri, P., Ciuti, G., Lucarini, G., Salerno, M., Di Natali, C., Verra, M., Dario, P. and Morino, M. (2013) 'Experimental assessment of a novel robotically driven endoscopic capsule compared to traditional colonoscopy', *Digestive and Liver Disease*, 45(8), pp. 657-662.

3. Banzi, M. and Shiloh, M. (2022) *Getting started with Arduino: the open-source electronics prototyping platform*. 4th edn. Maker Media, Inc.

4. Bluepad32 Contributors (2023) *Bluepad32 Documentation*. Available at: https://github.com/ricardoquesada/bluepad32 (Accessed: 15 April 2025).

5. Bro-Nielsen, M. and Cotin, S. (1996) 'Real-time volumetric deformable models for surgery simulation', *IEEE Transactions on Visualization and Computer Graphics*, 2(1), pp. 62-73.

6. Carpi, F., Kastelein, N., Talcott, M. and Pappone, C. (2010) 'Magnetically controllable gastrointestinal steering of video capsules', *IEEE Transactions on Biomedical Engineering*, 58(2), pp. 231-234.

7. Ciuti, G., Donlin, R., Valdastri, P., Arezzo, A., Menciassi, A., Morino, M. and Dario, P. (2010) 'Robotic versus manual control in magnetic steering of an endoscopic capsule', *Endoscopy*, 42(02), pp. 148-152.

8. Coles, T.R., Meglan, D. and John, N.W. (2011) 'The role of haptics in medical training simulators: A survey of the state of the art', *IEEE Transactions on Haptics*, 4(1), pp. 51-66.

9. Dharshan Keba Morris (2025) *Open-Source Control Platform for a Medical Robot Using Standard Components*. MSc Biomedical Engineering, Interim Report. University of Dundee.

10. Dharshan Keba Morris (DHARSHAN 212) (2025) *Source Code for Open-Source Colonoscopy Control Platform*. Available at: https://github.com/Dharshan212/OPEN-SOURCE-CONTROL-PLATFORM-FOR-MEDICAL-ROBOT

11. Diller, E., Giltinan, J., Lum, G.Z., Ye, Z. and Sitti, M. (2016) 'Six-degree-of-freedom magnetic actuation for wireless micro robotics', *International Journal of Robotics Research*, 35(1-3), pp. 114-128.

12. Edelmann, J., Petruska, A.J. and Nelson, B.J. (2018) 'Estimation-based control of a magnetic endoscope without device localization', *Journal of Medical Robotics Research*, 3(02), p. 1850002.

13. Espressif Systems (2023) *ESP32 Technical Reference Manual*. Available at: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf (Accessed: 17 June 2025).

14. Fedorov, A., Beichel, R., Kalpathy-Cramer, J., Finet, J., Fillion-Robin, J.C., Pujol, S., Bauer, C., Jennings, D., Fennessy, F., Sonka, M. and Buatti, J. (2012) '3D Slicer as an image computing platform for the Quantitative Imaging Network', *Magnetic Resonance Imaging*, 30(9), pp. 1323-1341.

15. Finocchiaro, M., Banfi, T., Donaire, S., Arezzo, A., Guarner-Argente, C., Menciassi, A., Casals, A., Ciuti, G. and Hermansanz, A. (2024) 'A Framework for the Evaluation of Human Machine Interfaces of Robot-Assisted Colonoscopy', *IEEE Transactions on Biomedical Engineering*, 71(3), pp. 987-998.

16. Gallagher, A.G., Ritter, E.M., Champion, H., Higgins, G., Fried, M.P., Moses, G., Smith, C.D. and Satava, R.M. (2005) 'Virtual reality simulation for the operating room: proficiency-based training as a paradigm shift in surgical skills training', *Annals of Surgery*, 241(2), pp. 364-372.

17. Ibanez, L., Schroeder, W., Ng, L. and Cates, J. (2005) *The ITK Software Guide*. 2nd edn. Kit ware, Inc.

18. Jamjoom, A.A., Jamjoom, A.M. and Marcus, H.J. (2020) 'Exploring public opinion about liability and responsibility in surgical robotics', *Nature Machine Intelligence*, 2(4), pp. 194-196.

19. Kikinis, R., Pieper, S.D. and Vosburgh, K.G. (2014) '3D Slicer: a platform for subject-specific image analysis, visualization, and clinical support', in Jolesz, F.A. (ed.) *Intraoperative Imaging and Image-Guided Therapy*. Springer, New York, NY, pp. 277-289.

20. Mahoney, A.W. and Abbott, J.J. (2016) 'Five-degree-of-freedom manipulation of an untethered magnetic device in fluid using a single permanent magnet with application in stomach capsule endoscopy', *International Journal of Robotics Research*, 35(1-3), pp. 129-147.

21. Martin, J.W., Scaglioni, B., Norton, J.C., Subramanian, V., Arezzo, A., Obstein, K.L. and Valdastri, P. (2020) 'Enabling the future of colonoscopy with intelligent and autonomous magnetic manipulation', *Nature Machine Intelligence*, 2, pp. 595-606.

22. MathWorks (2023) *MATLAB and Simulink for Medical Devices*. Available at: https://www.mathworks.com/solutions/medical-device.html (Accessed: 7 February 2025).

23. Mu, Y., Yang, X., Guo, F., Ye, G., Lu, Y., Zhang, Y., Xue, W. and Bian, Z. (2024) 'Colonoscopy Training on Virtual-Reality Simulators or Physical Model Simulators: A Randomized Controlled Trial', *Journal of Surgical Education*, 81(10), pp. 1339-1345.

24. Okamura, A.M. (2004) 'Haptic feedback in robot-assisted minimally invasive surgery', *The International Journal of Robotics Research*, 23(10-11), pp. 1203-1217.

25. Open-Source Initiative (2023) *The MIT License*. Available at: https://opensource.org/licenses/MIT (Accessed: 15 July 2025).

26. Pavlosky, E.N. (1960) *Human diseases with natural foci*. Foreign Language Publishing House, Moscow.

27. Pittiglio, G., Barducci, L., Martin, J.W., Norton, J.C., Obstein, K.L. and Valdastri, P. (2019) 'Magnetic Levitation for Soft-Tethered Capsule Colonoscopy Actuated with a Single Permanent Magnet: A Dynamic Control Approach', *IEEE Robotics and Automation Letters*, 4(2), pp. 1224-1231.

28. Prendergast, J.M., Formosa, G.A., Heckman, C.R. and Rentschler, M.E. (2018) 'Towards autonomous control of flexible endoscopes for natural orifice transluminal endoscopic surgery', in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Madrid, Spain, October 2018. IEEE, pp. 783-790.

29. Richardson, M. and Wallace, S. (2015) *Getting Started with Raspberry Pi*. 3rd edn. O'Reilly Media.

30. Scaglioni, B., Martin, J.W., Norton, J.C., Obstein, K.L. and Valdastri, P. (2019) 'Explicit Model Predictive Control of a Magnetic Flexible Endoscope', *IEEE Robotics and Automation Letters*, 4(2), pp. 716-723.

31. Seymour, N.E., Gallagher, A.G., Roman, S.A., O'Brien, M.K., Bansal, V.K., Andersen, D.K. and Satava, R.M. (2002) 'Virtual reality training improves operating room performance: results of a randomized, double-blinded study', *Annals of Surgery*, 236(4), pp. 458-464.

32. Shamsudhin, N., Zverev, V.I., Keller, H., Pane, S., Haber, T., Egolf, P.W., Nelson, B.J. and Tishin, A.M. (2017) 'Magnetically guided capsule endoscopy', *Medical Physics*, 44(5), pp. e91-e111.

33. Sikorski, J., Denasi, A., Biondi, G., Scheggi, S. and Misra, S. (2019) 'Vision-Based 3-D Control of Magnetically Actuated Catheter Using BigMag—An Array of Mobile Electromagnetic Coils', *IEEE/ASME Transactions on Mechatronics*, 24(2), pp. 505-516.

34. Slavinski, R.R., Obstein, K.L. and Valdastri, P. (2015) 'Emerging issues and future developments in capsule endoscopy', *Techniques in Gastrointestinal Endoscopy*, 17(1), pp. 40-46.

35. Son, D., Gilbert, H. and Sitti, M. (2020) 'Magnetically Actuated Soft Capsule Endoscope for Fine-Needle Biopsy', *Soft Robotics*, 7(1), pp. 10-21.

36. Taddees, A.Z., Slavinski, R.R., Obstein, K.L. and Valdastri, P. (2016) 'Nonholonomic closed-loop velocity control of a soft-tethered magnetic capsule endoscope', in *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Daejeon, Korea (South), October 2016. IEEE, pp. 1159-1164.

37. Taylor, Z.A., Cheng, M. and Ourselin, S. (2008) 'Real-time nonlinear finite element analysis for surgical simulation using graphics processing units', in *Medical Image Computing and Computer-Assisted Intervention–MICCAI 2008: 11th International Conference, New York, NY, USA, September 6-10, 2008, Proceedings, Part I 11*. Springer, pp. 701-708.

38. Valdastri, P., Simi, M. and Webster III, R.J. (2012) 'Advanced technologies for gastrointestinal endoscopy', *Annual Review of Biomedical Engineering*, 14, pp. 397-429.

39. Vassiliou, M.C., Dunkin, B.J., Marks, J.M. and Fried, G.M. (2010) 'Fundamentals of endoscopic surgery: creation and validation of the hands-on test', *Surgical Endoscopy*, 24(8), pp. 1836-1841.

40. Wickens, C.D., Hollands, J.G., Banbury, S. and Parasuraman, R. (2015) *Engineering Psychology & Human Performance*. 4th edn. Psychology Press.

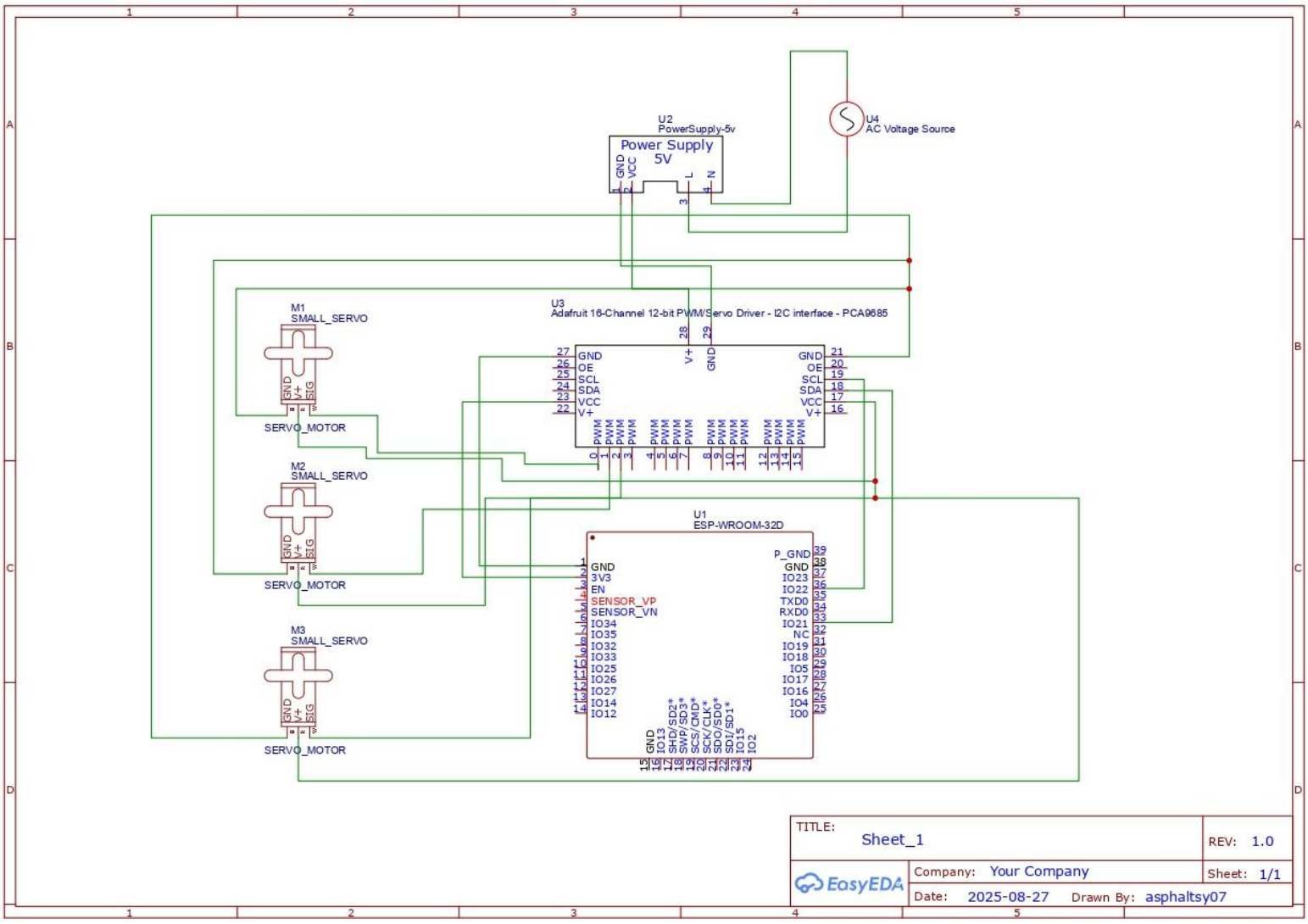
41. Xu, T., Zhang, J., Salehizadeh, M., Onaizah, O. and Diller, E. (2019) 'Millimeter-scale flexible robots with programmable three-dimensional magnetization and motions', *Science Robotics*, 4(29), p. eaav4494.

42. Yang, G.Z., Cambias, J., Cleary, K., Daimler, E., Drake, J., Dupont, P.E., Hata, N., Kazanzides, P., Martel, S., Patel, R.V. and Santos, V.J. (2017) 'Medical robotics—Regulatory, ethical, and legal considerations for increasing levels of autonomy', *Science Robotics*, 2(4), p. eaam8638.

APPENDICES

APPENDIX A

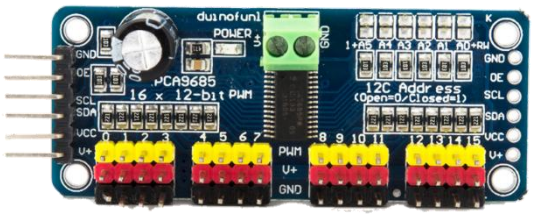
Hardware Schematics

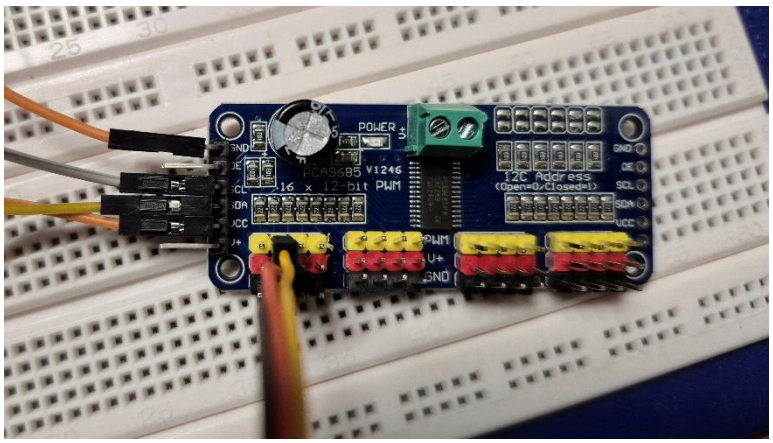


APPENDIX B

Pictures of Hardware

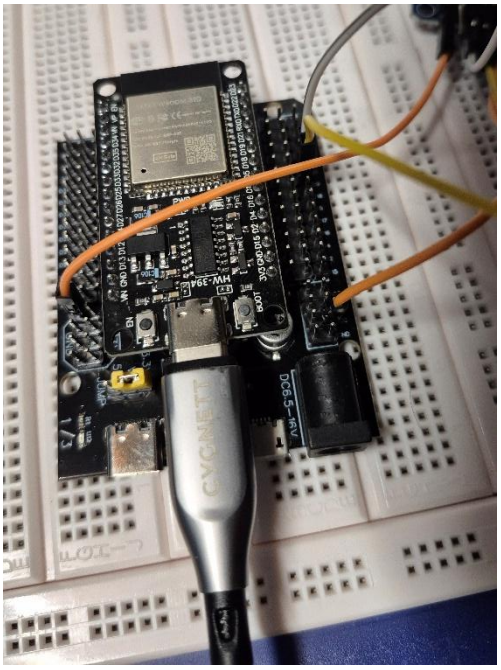
PCA9685





ESP32 – WROOM 32D





CONTINIOUS SERVO FS90R



